

A robotic arm with orange and black segments, labeled 'WLKATA', is positioned over a chessboard. The arm's gripper is holding a black chess piece, likely a pawn, and is about to move it. The chessboard is set up with various pieces, including pawns, knights, and a king. In the background, there is a framed abstract painting with a large orange circle and a smaller grey circle. To the right, a small potted plant with green leaves is visible. The scene is set on a light-colored surface.

Reinforcement learning



Topics



Intro



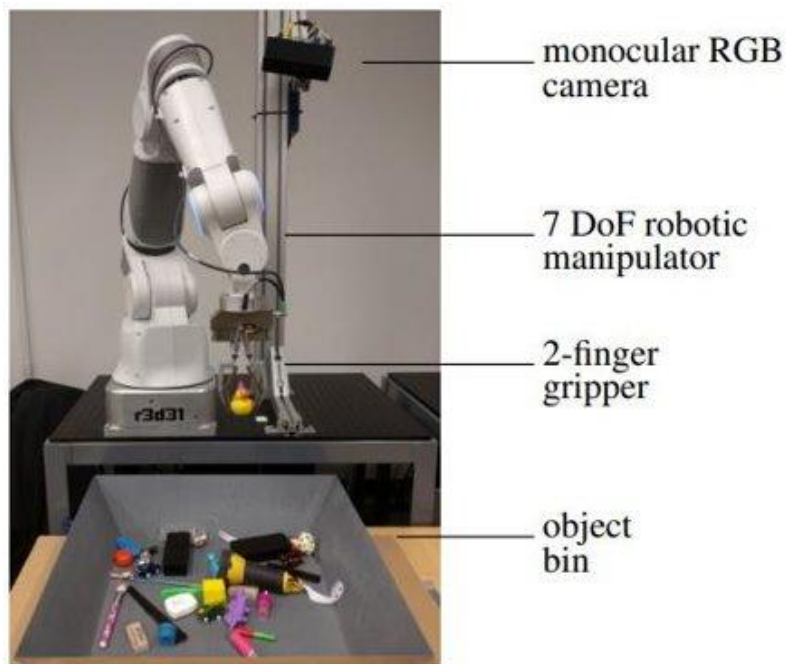
Q-learning



Deep Q-learning



Policy Gradient



Problem: Controller for sorting robot



Input

Controller

Move to (x,y,z)

Option 1: Understand the problem, design a solution



Option 2: Set it up as a machine learning problem

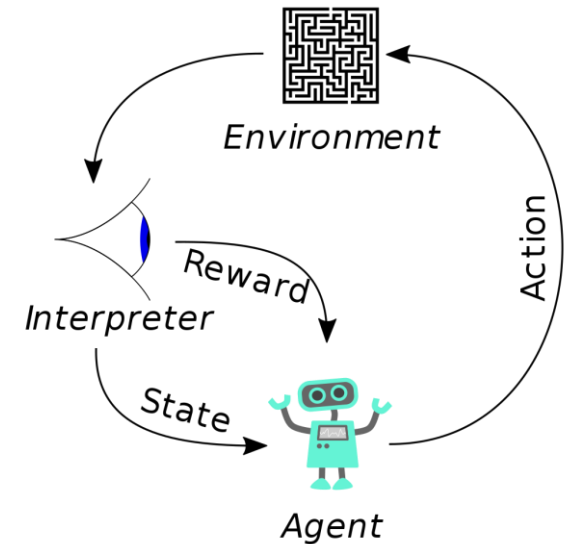


ML Controller

Move to (x,y,z)

What is Reinforcement Learning?

- Reinforcement learning is a type of Machine Learning where an **agent** learns how to behave in an **environment** by performing **actions** and maximize the notion of cumulative **reward**.
- ~How humans learn:
 - Doing something good -> positive reward
 - Doing something bad -> negative reward



Reinforcement Learning

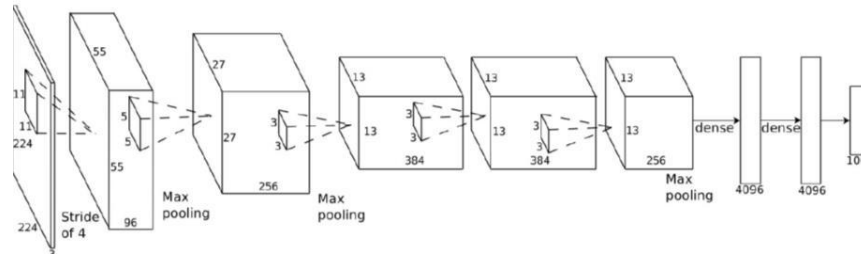


Paper from UC Berkeley: Wu, et al. “DayDreamer: World Models for Physical Robot Learning”, 2022

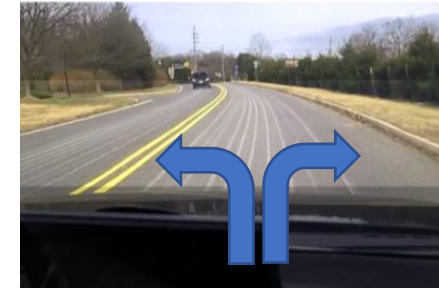
Imitation Learning (Supervised learning behavior) (not RL)



Observation/state $\mathbf{o}_t$



Policy $\pi_{\theta}(\mathbf{a}_t|\mathbf{o}_t)$



Action $\mathbf{a}_t$



$\mathbf{o}_t$

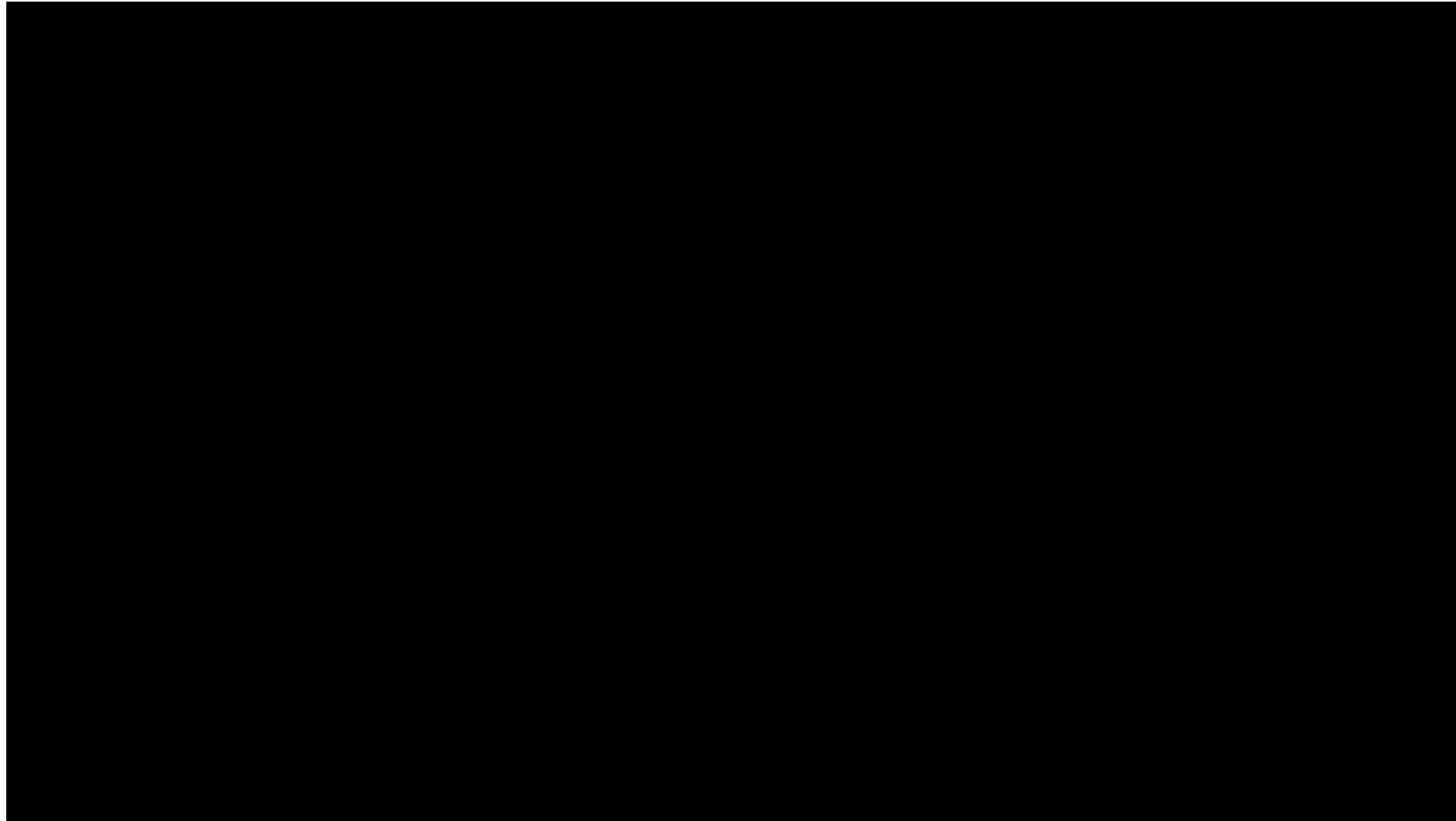
$\mathbf{a}_t$



Supervised
learning

$\pi_{\theta}(\mathbf{a}_t|\mathbf{o}_t)$

Imitation Learning (Supervised learning behaviour) (not RL)



Quadcopter Navigation in the Forest using Deep Neural Networks



AAAI Video Competition
2.47K subscribers

Subscribe

1.9K | 7 | Share | Download | Clip | Save | ...

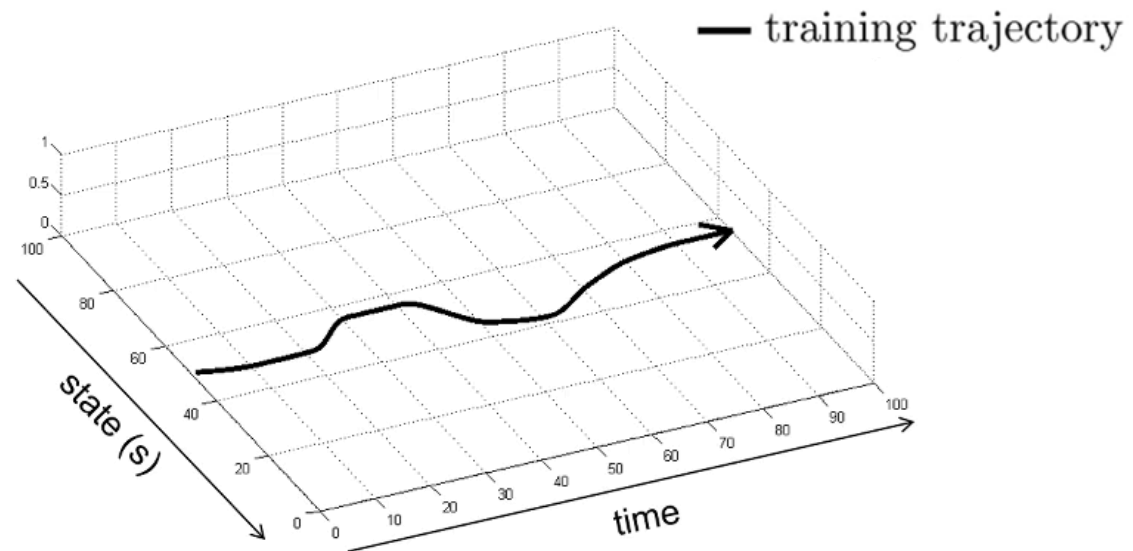
142K views · 8 years ago

We study the problem of perceiving forest or mountain trails from a single monocular image acquired from the viewpoint of a robot traveling on the trail itself. Previous literature focused on trail segmentation, and used low-level features such as image saliency or appearance contrast; we propose a different approach based on a Deep Neural Network used as a supervised image classifier. By operating on the whole image at once, our system outputs the main direction of the trail compared to the viewing direction. Qualitative and quantitative results computed on a large real-world data set ...more

<https://youtu.be/umRdt3zGgpU>

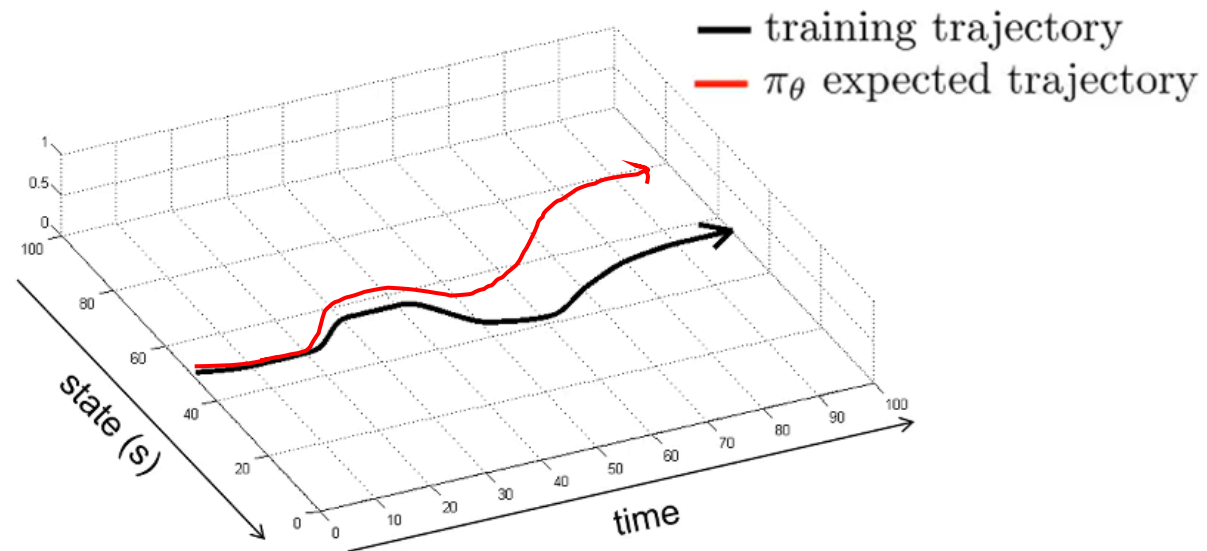
Imitation Learning (Supervised learning behaviour) (not RL)

- Will it work always?



Imitation Learning (Supervised learning behaviour) (not RL)

- Will it work always?



Imitation Learning (Supervised learning behaviour) (not RL)

Multi-lane highway driving

Entire clip is autodriven except for the lane change

Imitation Learning (Supervised learning behaviour) (not RL)

- Bojarski et al. “End to End Learning for Self-Driving Cars”, 2016.

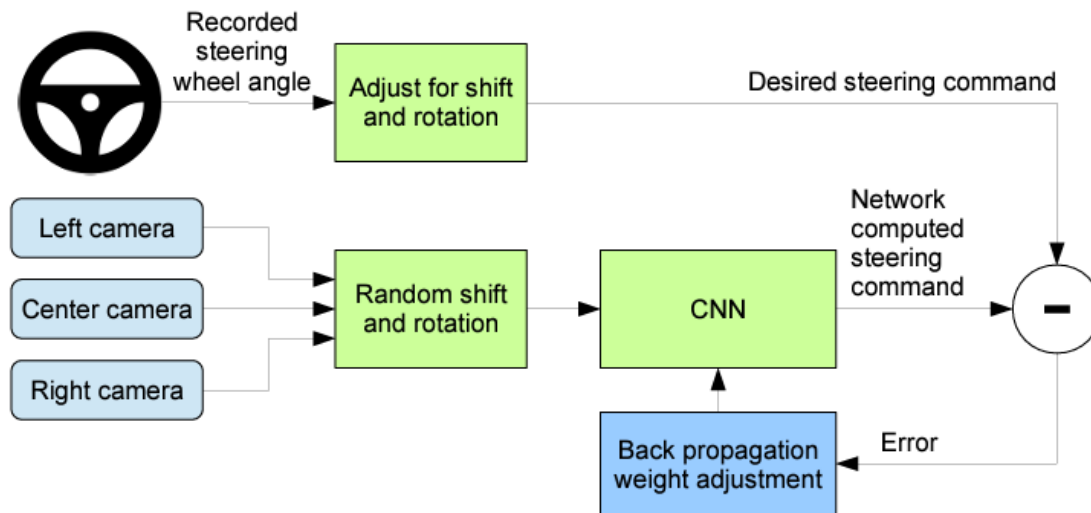


Figure 2: Training the neural network.



Imitation Learning (Supervised learning behaviour) (not RL)

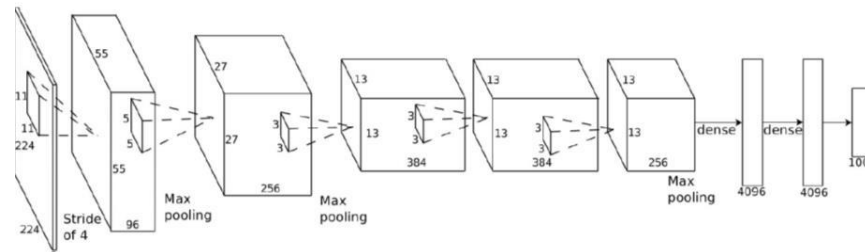
- Can we make it work more often?

Dagger: Dataset Aggregation

- Train $\pi_{\theta}(\mathbf{a}_t|\mathbf{o}_t)$ from human data $D = \{o_1, a_1, o_2, a_2, \dots\}$
- Run $\pi_{\theta}(\mathbf{a}_t|\mathbf{o}_t)$ the network to get data set $D_{\theta} = \{o_1, o_2, \dots\}$
- Ask human to label D_{θ} with actions a_{θ}
- Aggregate: $D \leftarrow D \cup D_{\theta}$
- Repeat enough times to have very low errors in $\pi_{\theta}(\mathbf{a}_t|\mathbf{o}_t)$



$\mathbf{o}_t$



$\pi_{\theta}(\mathbf{a}_t|\mathbf{o}_t)$



$\mathbf{a}_t$

Imitation Learning (Supervised learning behaviour) (not RL)

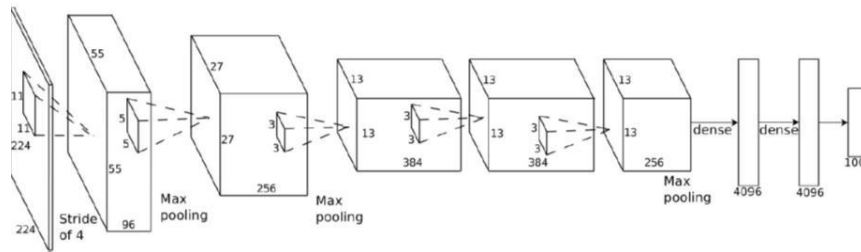
- Problem...

Dagger: Dataset Aggregation

- Train $\pi_{\theta}(\mathbf{a}_t|\mathbf{o}_t)$ from human data $D = \{o_1, a_1, o_2, a_2, \dots\}$
- Run $\pi_{\theta}(\mathbf{a}_t|\mathbf{o}_t)$ the network to get data set $D_{\theta} = \{o_1, o_2, \dots\}$
- Ask human to label D_{θ} with actions a_{θ}
- Aggregate: $D \leftarrow D \cup D_{\theta}$
- Repeat enough times to have very low errors in $\pi_{\theta}(\mathbf{a}_t|\mathbf{o}_t)$



$\mathbf{o}_t$



$\pi_{\theta}(\mathbf{a}_t|\mathbf{o}_t)$

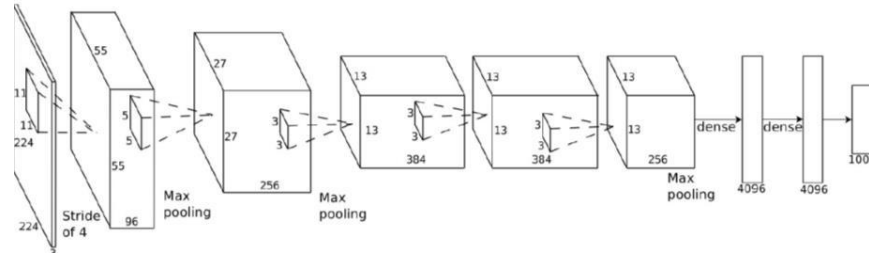


$\mathbf{a}_t$

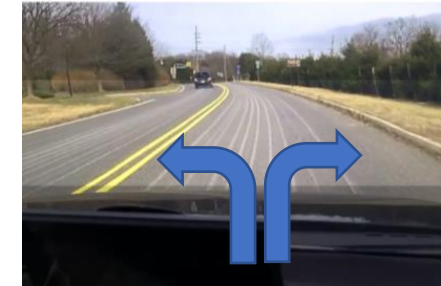
Reward functions in Reinforcement learning



Observation $\mathbf{o}_t$



$$\pi_{\theta}(\mathbf{a}_t | \mathbf{o}_t)$$



Action $\mathbf{a}_t$

Which action is better or worse?

$r(s, a)$: Reward function at state s , with action a .

Tells us which state and action are better

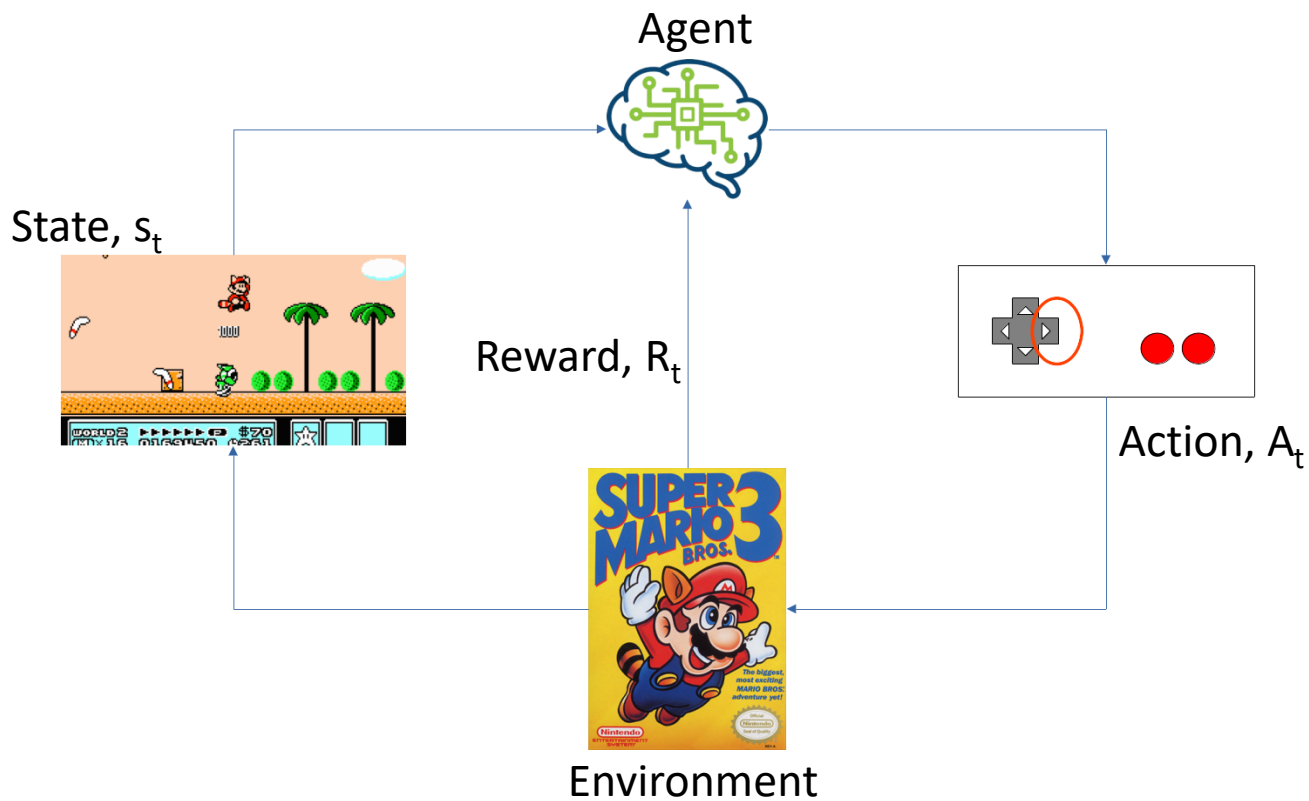


High reward



Low reward

The General Reinforcement Learning Process



- The **agent** receives **state** S_t at time t from the **Environment** (In this case we receive the first frame of our game (state) from Super Mario Bros (environment))
- Based on that **state** S_t , the agent takes an **action** A_t (our agent will move right)
- Environment transitions to a **new state** $S_{(t+1)}$ (new frame)
- Environment gives some **reward** $R_{(t+1)}$ to the agent (not dead: +1)

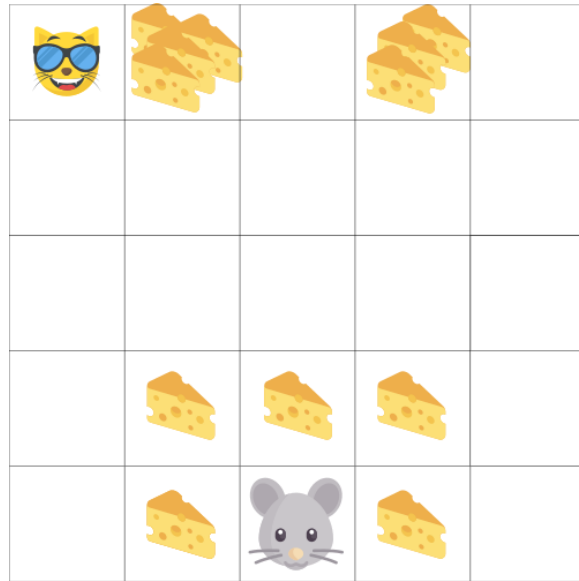
The goal of the agent is to choose actions that maximize the expected cumulative reward.

$$G_t = R_{t+1} + R_{t+2} + R_{t+3} + \dots$$

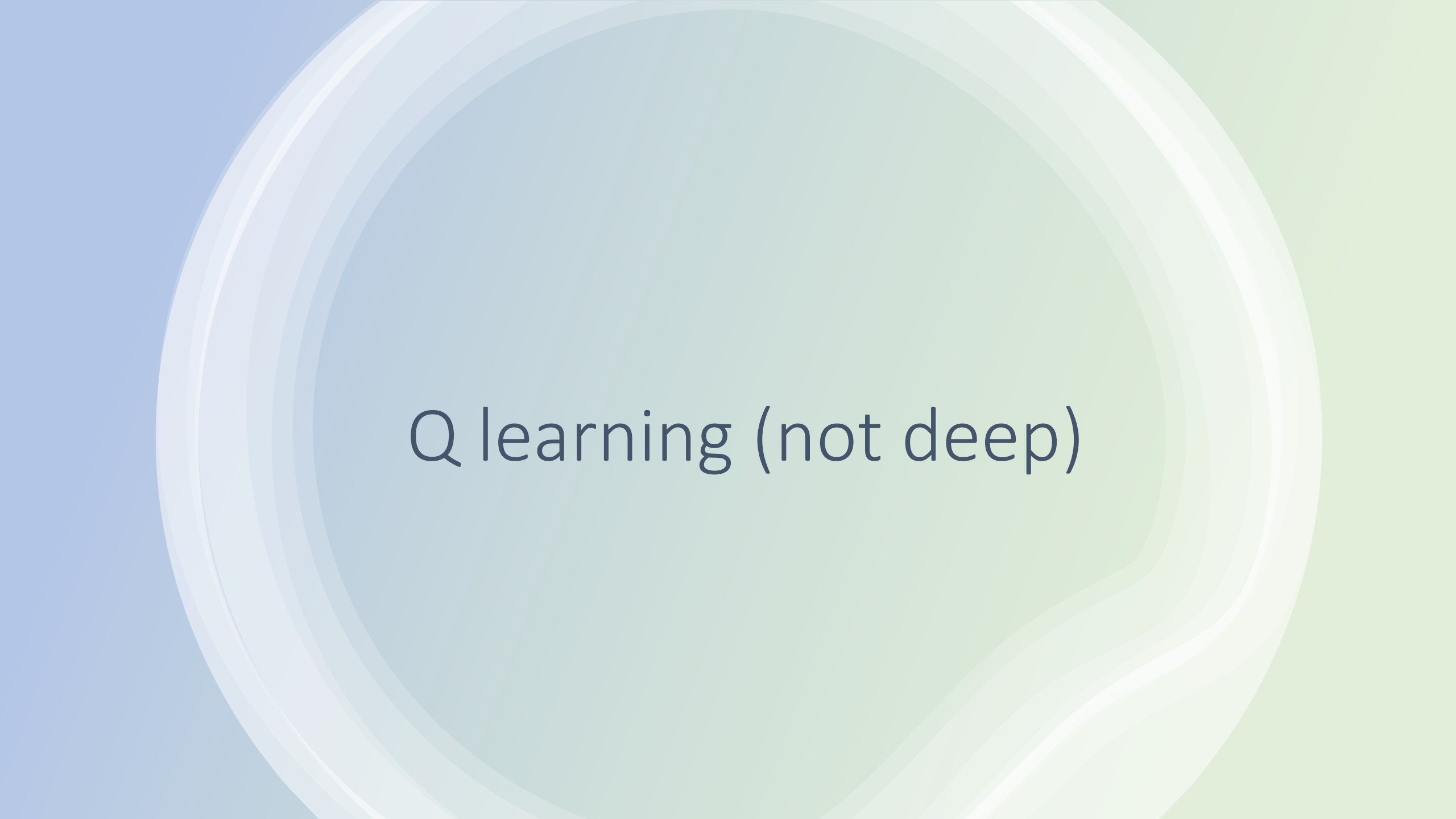
However, the reward that come sooner is more likely to happen and should, therefore, have more weight. Therefore, we often multiply a discount, γ , to the future reward.

$$G_t = \gamma R_{t+1} + \gamma^2 R_{t+2} + \gamma^3 R_{t+3} + \dots \quad \text{where} \quad 0 \leq \gamma \leq 1$$

Discounted expected reward example: cat vs mouse



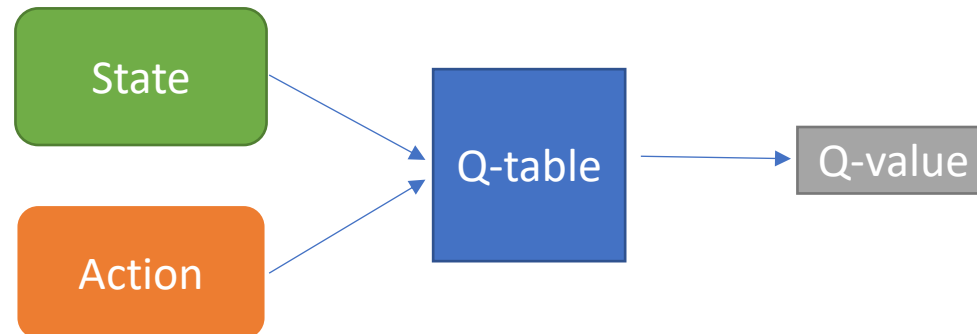
- Let say your agent is this small mouse and your opponent is the cat. Your goal is to eat the maximum amount of cheese before being eaten by the cat.
- The rewards that come sooner (in the beginning of the game) are more probable to happen, since they are more predictable than the long-term future reward.



Q learning (not deep)

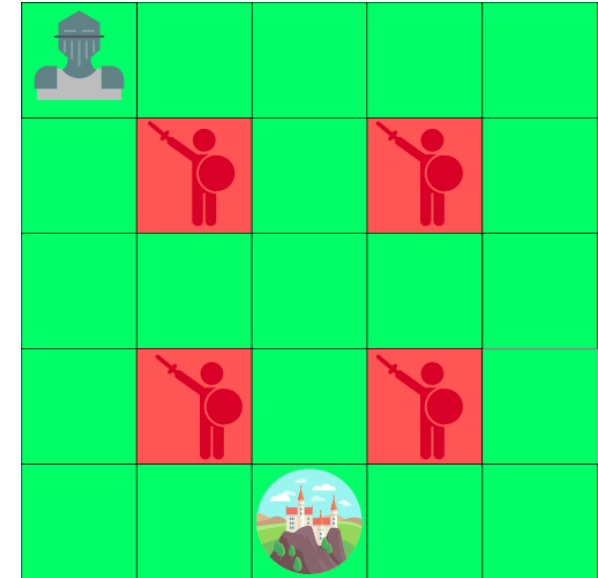
Q-learning

- Problem: selecting an action based on the current state
- In Q-learning, we create a Q table that helps us find what action to take for each state.
 - Q = quality of an action
- The goal is to maximize the value function Q, which is the expected future reward given a state and action.
- With a state and a trained Q-table, we can choose new actions



Q-learning

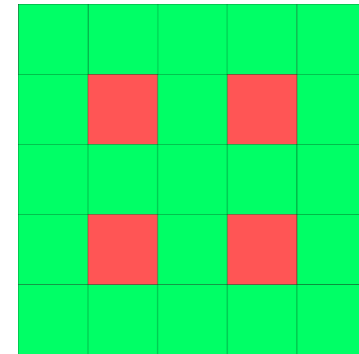
- Let's say we want to move the knight to the castle but avoid the enemies, but we don't know where the enemies are
- We could create a reward system:
 - Each step costs -1 points
 - Touching the enemy costs -100 points and the episode ends
 - Touching the castle and you get $+100$ points



Green is safe, red is not safe

Q-learning

We do not know which neighbouring green tile that is the best choice and we could end up going back and forth.



Green is safe, red is not safe

Q Table

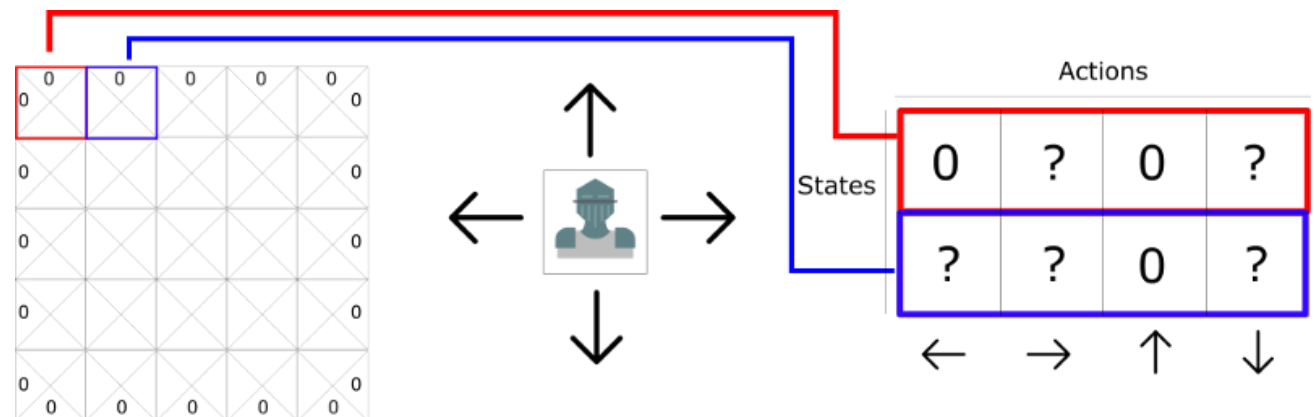
To solve the problem, we can create a Q-table that will tell the maximum expected future reward, for each action at each state. I.e., the expected best move.

Think of this Q-table as a game “cheat sheet.” Thanks to that, we know for each state (each line in the Q-table) what’s the best action to take, by finding the highest score in that line.

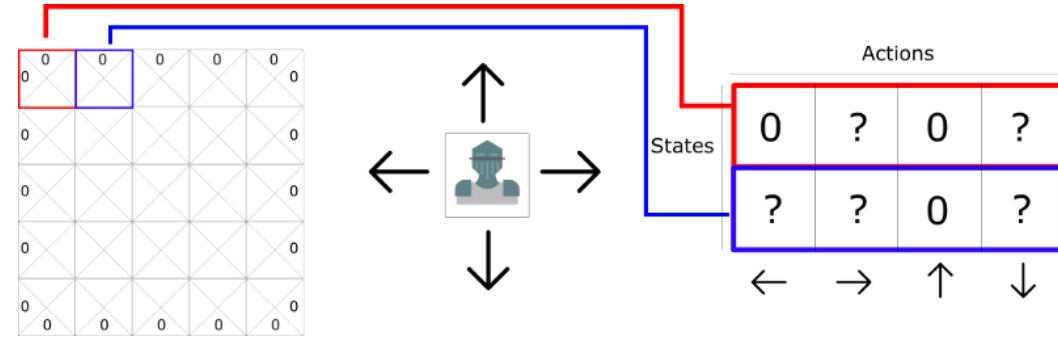
The table's content will be learned from experience.

Columns are possible actions

Rows are possible states



Q-learning algorithm (1/2)



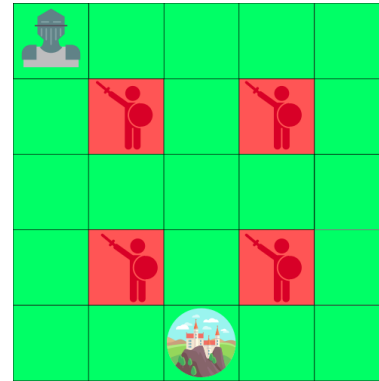
$$Q(s_t, a_t) = \mathbb{E}(R_{t+1} + \gamma^1 R_{t+2} + \gamma^2 R_{t+3} + \dots | s_t, a_t)$$

Q-value for stage given action

Expected cumulative reward

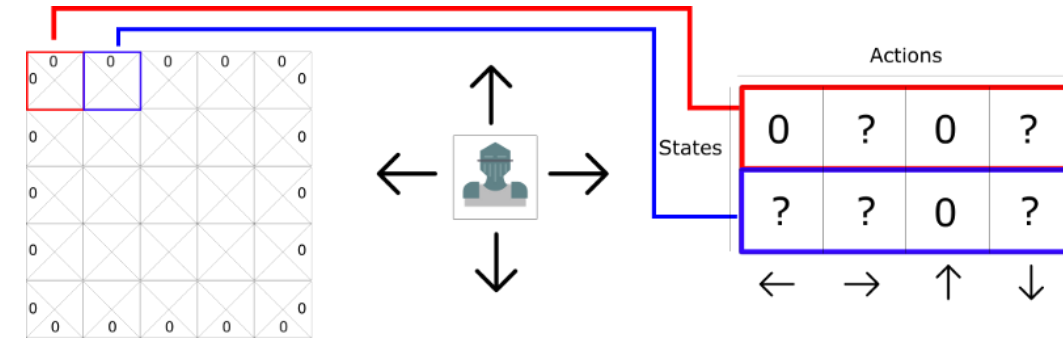
Given state and action

- In the q-table, you will find the row corresponding to the current state and select the action of the column with the highest value.



Q-learning algorithm (2/2)

In the beginning, the Q-value is 0 for all actions. We need to explore the environment to create the Q-table.

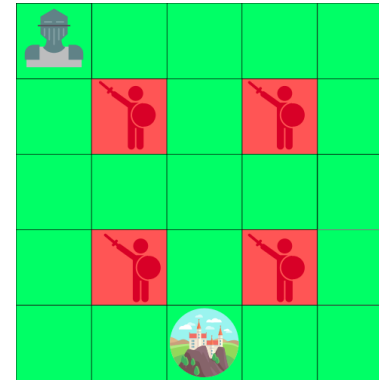


$$Q(s_t, a_t) = \mathbb{E} (R_{t+1} + \gamma^1 R_{t+2} + \gamma^2 R_{t+3} + \dots | s_t, a_t)$$

Q-value for state given action

Expected cumulative reward

Given state and action



Initialize Q-table to zeros (unknown world)

Choose an action a at state s based on Q-value estimates $Q(s, a)$

Perform action a

Measure reward R and outcome state s'

Update Q using Bellmans' eq.

Good Q-table

Repeat

Finally

a = Action that maximizes expected future reward

Bellman equation:

Reward for that action a at state s

Maximum expected future reward given new state s' and all possible actions a' at that new state

Current Q value

$$Q(s, a) := Q(s, a) + \alpha [R(s, a) + \gamma \max_{a'} Q(s', a') - Q(s, a)]$$

New Q for that state and action

Learning rate

Discount rate

Q-learning: example: step 0

- One cheese = +1
- Two cheeses = +2
- Big pile of cheese = +10 (end of the episode)
- If you eat rat poison = -10 (end of the episode)



Initial Q-table (only possible moves)

State	←	→	↑	↓
Start	x	0	x	0
1 cheese	0	0	x	0
Empty	0	x	x	0
Two small cheeses	x	0	0	0
Poison	0	0	0	x
Big pile of cheese	0	x	0	x

Q-learning: example: step 1

- One cheese = +1
- Two cheeses = +2
- Big pile of cheese = +10 (end of the episode)
- If you eat rat poison = -10 (end of the episode)

Initial Q-table (only possible moves)

State	←	→	↑	↓
Start		0		0
1 cheese	0	0		0
Empty	0			0
Two small cheeses		0	0	0
Poison	0	0	0	
Big pile of cheese	0		0	



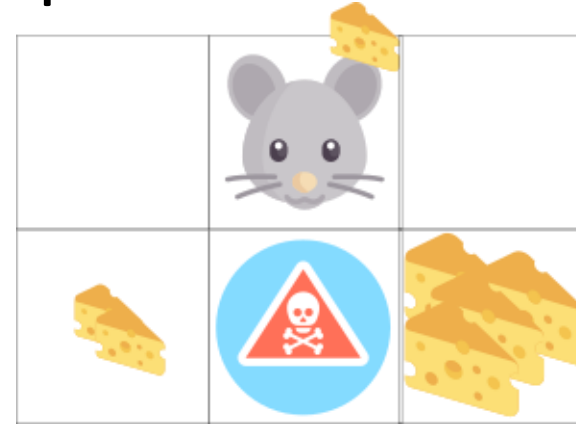
Action: choose action of highest Q-value. Since they are equal, we choose randomly and move right

Q-learning: example: step 1

- One cheese = +1
- Two cheeses = +2
- Big pile of cheese = +10 (end of the episode)
- If you eat rat poison = -10 (end of the episode)

Initial Q-table (only possible moves)

State	←	→	↑	↓
Start		0		0
1 cheese	0	0		0
Empty	0			0
Two small cheeses		0	0	0
Poison	0	0	0	
Big pile of cheese	0		0	



Reward: We find one cheese (!) and get a reward of +1

$$\text{Bellman equation: } Q(s, a) := Q(s, a) + \alpha \left[R(s, a) + \gamma \max_{a'} Q(s', a') - Q(s, a) \right]$$

Q-learning: example: step 1

- One cheese = +1
- Two cheeses = +2
- Big pile of cheese = +10 (end of the episode)
- If you eat rat poison = -10 (end of the episode)



Initial Q-table (only possible moves)

State	←	→	↑	↓
Start		0		0
1 cheese	0	0		0
Empty	0			0
Two small cheeses		0	0	0
Poison	0	0	0	
Big pile of cheese	0		0	

Update Q-value:

a' is possible moves at new state (1 cheese).

i.e., $\leftarrow \downarrow \rightarrow$

$$Q(\text{start}, \rightarrow) := \underbrace{Q(\text{start}, \rightarrow)}_{=0} + \underbrace{\alpha}_{=0.1} \left[\underbrace{R(\text{start}, \rightarrow)}_{=+1} + \underbrace{\gamma}_{=0.9} \underbrace{\max_{a'} Q(1\text{cheese}, a')}_{=0} - \underbrace{Q(\text{start}, \rightarrow)}_{=0} \right]$$

$$Q(\text{start}, \rightarrow) := 0 + 0.1 [1 + 0.9 \cdot 0 - 0] = 0.1$$

Bellman equation: $Q(s, a) := Q(s, a) + \alpha \left[R(s, a) + \gamma \max_{a'} Q(s', a') - Q(s, a) \right]$

Q-learning: example: step 1

- One cheese = +1
- Two cheeses = +2
- Big pile of cheese = +10 (end of the episode)
- If you eat rat poison = -10 (end of the episode)



Initial Q-table (only possible moves)

State	←	→	↑	↓
Start		0.1		0
1 cheese	0	0		0
Empty	0			0
Two small cheeses		0	0	0
Poison	0	0	0	
Big pile of cheese	0		0	

Update Q-value:

a' is possible moves at new state (1 cheese).

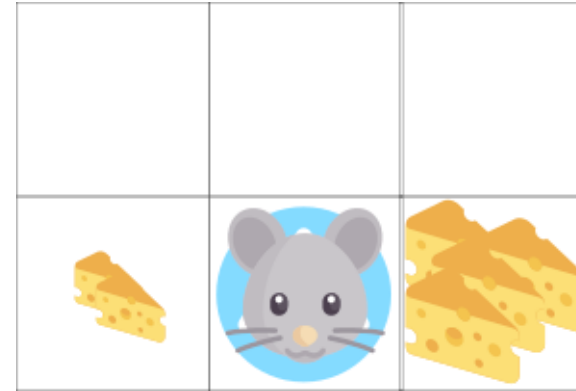
i.e., $\leftarrow \downarrow \rightarrow$

$$Q(start, \rightarrow) := \underbrace{Q(start, \rightarrow)}_{=0} + \underbrace{\alpha}_{=0.1} \left[\underbrace{R(start, \rightarrow)}_{=+1} + \gamma \underbrace{\max_{a'} Q(1cheese, a')}_{=0.9} - \underbrace{Q(start, \rightarrow)}_{=0} \right]$$

$$Q(start, \rightarrow) := 0 + 0.1 [1 + 0.9 \cdot 0 - 0] = 0.1$$

Q-learning: example: step 2

- One cheese = +1
- Two cheesess = +2
- Big pile of cheese = +10 (end of the episode)
- If you eat rat poison = -10 (end of the episode)



Q-table (only possible moves)

State	←	→	↑	↓
Start		0.1		0
1 cheese	0	0		0
Empty	0			0
Two small cheeses		0	0	0
Poison	0	0	0	
Big pile of cheese	0		0	

Action: choose action of highest Q-value. Since they are equal, we choose randomly and move down

Q-learning: example: step 2

- One cheese = +1
- Two cheeses = +2
- Big pile of cheese = +10 (end of the episode)
- If you eat rat poison = -10 (end of the episode)



Q-table (only possible moves)

State	←	→	↑	↓
Start		0.1		0
1 cheese	0	0		0
Empty	0			0
Two small cheeses		0	0	0
Poison	0	0	0	
Big pile of cheese	0		0	

Reward: We find poison and get -10

Bellman equation: $Q(s, a) := Q(s, a) + \alpha \left[R(s, a) + \gamma \max_{a'} Q(s', a') - Q(s, a) \right]$

Q-learning: example: step 2

- One cheese = +1
- Two cheeses = +2
- Big pile of cheese = +10 (end of the episode)
- If you eat rat poison = -10 (end of the episode)

Q-table (only possible moves)

State	←	→	↑	↓
Start		0.1		0
1 cheese	0	0		-1
Empty	0			0
Two small cheeses		0	0	0
Poison	0	0	0	
Big pile of cheese	0		0	



Update Q:

a' is possible moves at new state.

$$Q(1cheese, \downarrow) := \underbrace{Q(1cheese, \downarrow)}_{=0} + \underbrace{\alpha}_{=0.1} \left[\underbrace{R(1cheese, \downarrow)}_{=-10} + \underbrace{\gamma}_{=0.9} \underbrace{\max_{a'} Q(poison, a')}_{=0} - \underbrace{Q(1cheese, \downarrow)}_{=0} \right]$$

$$Q(1cheese, \downarrow) := 0 + 0.1 [-10 + 0.9 \cdot 0 - 0] = -1$$

Because episode ends
when rat eats poison

Q-learning: example: New round 0

- One cheese = +1
- Two cheeses = +2
- Big pile of cheese = +10 (end of the episode)
- If you eat rat poison = -10 (end of the episode)

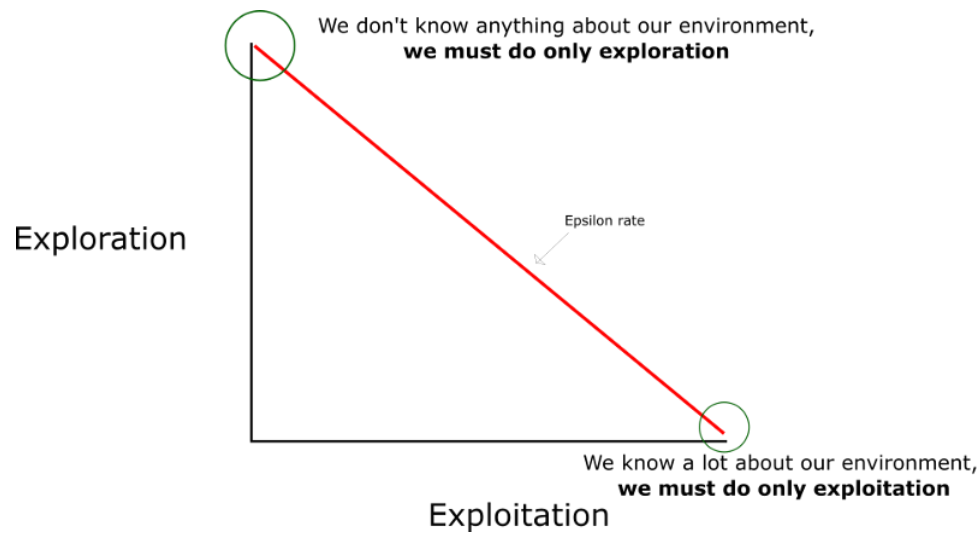


Initial Q-table (only possible moves)

State	←	→	↑	↓
Start		0.1		0
1 cheese	0	0		-1
Empty	0			0
Two small cheeses		0	0	0
Poison	0	0	0	
Big pile of cheese	0		0	

Exploration and exploitation

In the beginning we would like to explore the environment, after several episodes, we would choose action based on experience.

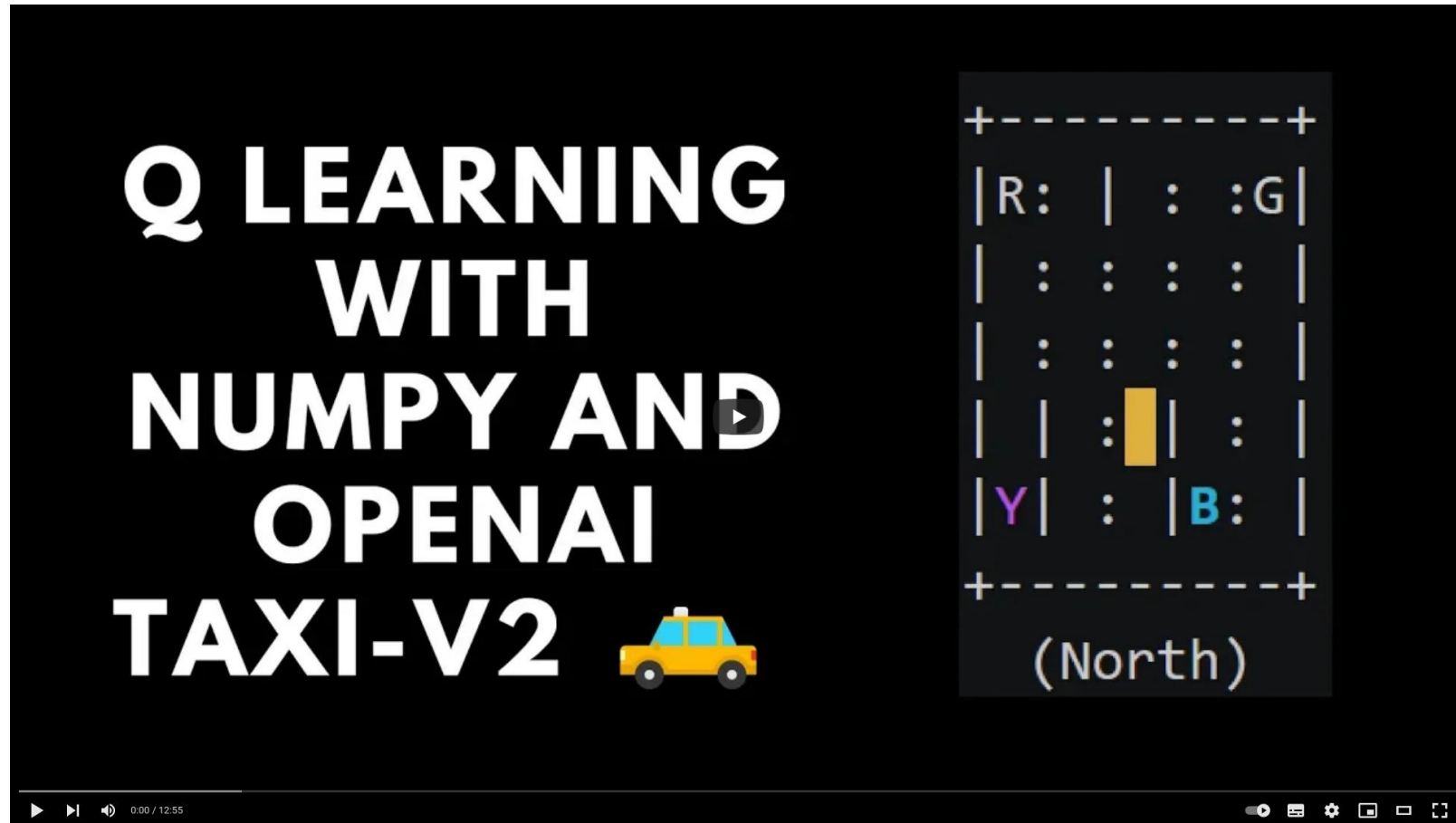


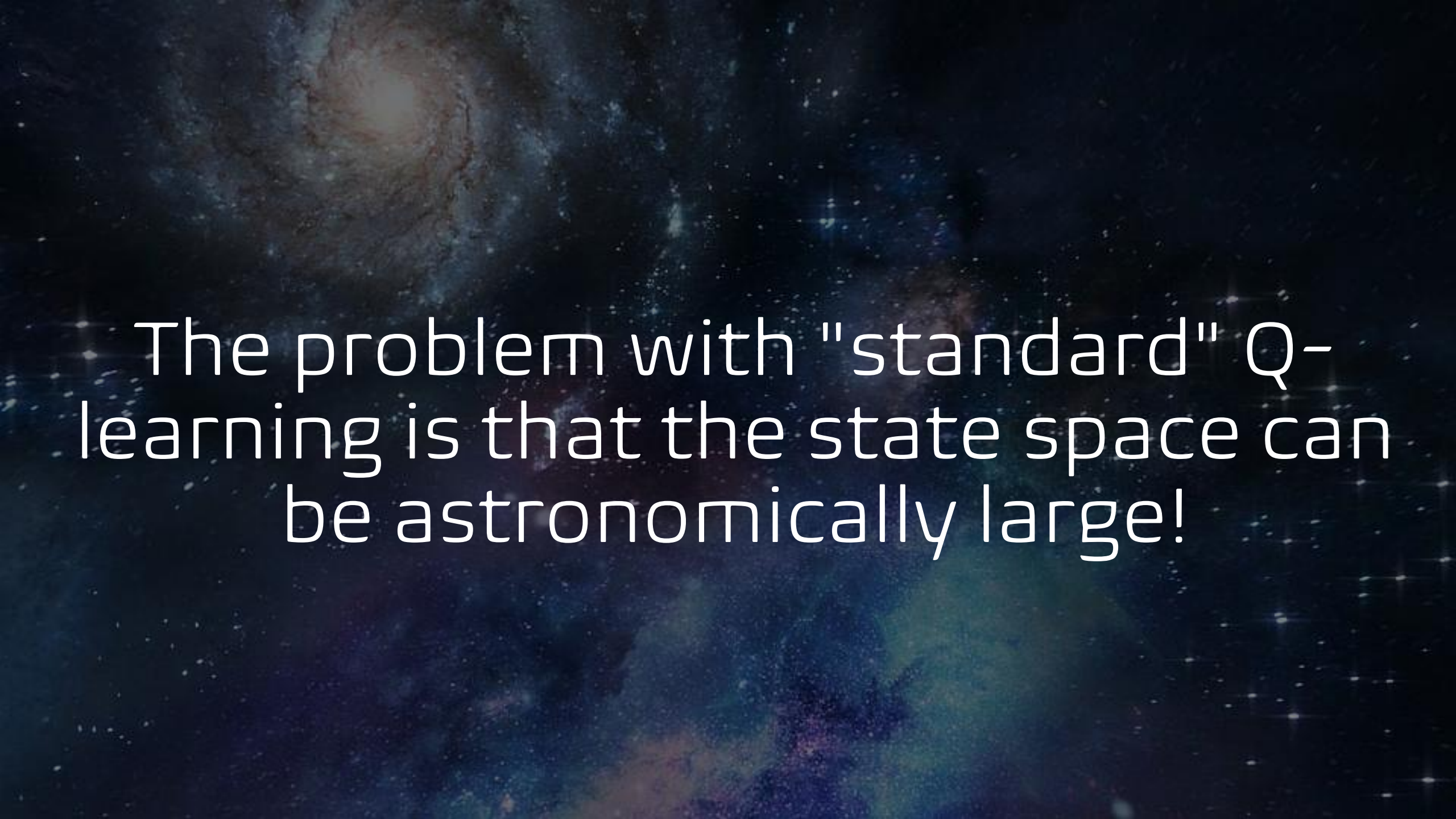
An exploration rate “epsilon” sets the probability of choosing a random action. It is set to 1 in the beginning, and it decays over time.

At each step, we choose a random number. If that number $< \epsilon$ we select a random action. Otherwise, we choose action based on the Q-table.

Q learning in numpy – Online tutorial

<https://youtu.be/q2ZOEFAaal0?t=152>

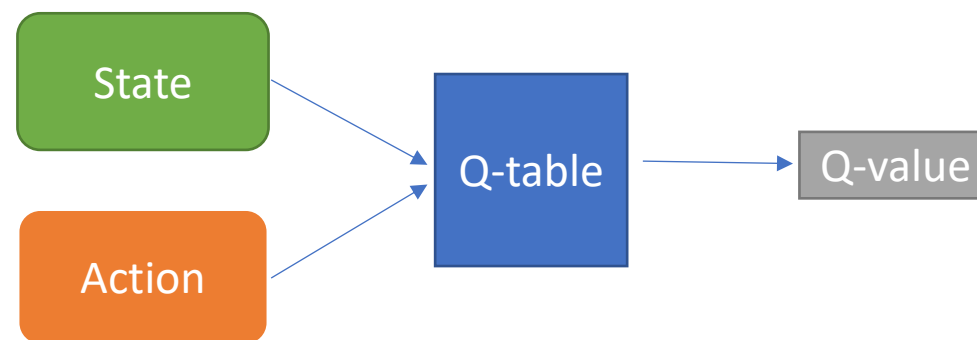




The problem with "standard" Q-learning is that the state space can be astronomically large!

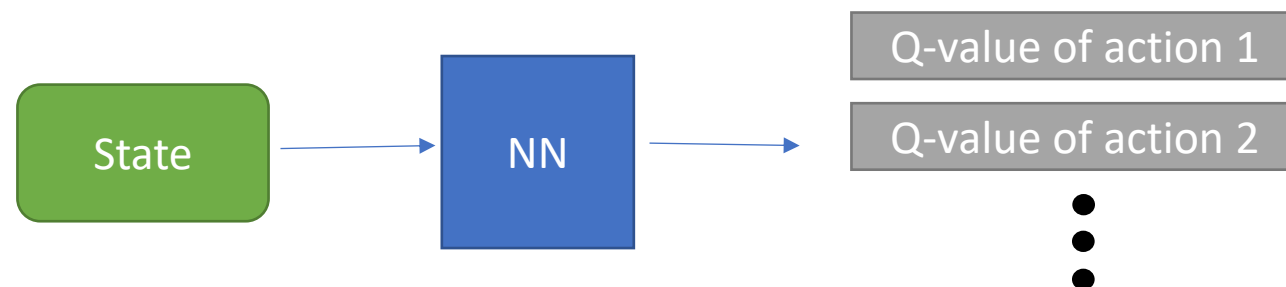
Q-learning

We create a table that tells the best new action given a state and action



Deep Q-learning

Instead of using a Q-table, we'll implement a Neural Network that takes a state and approximates Q-values for each action based on that state.



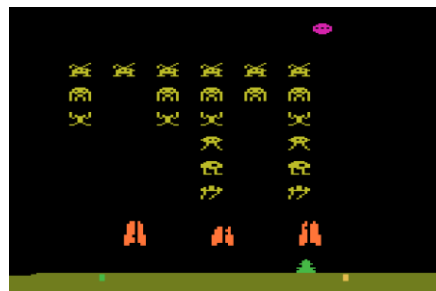
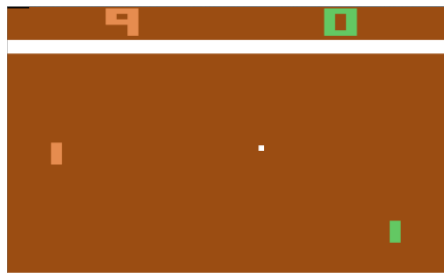
Deep Q-learning History

In 2013, Deep Mind modified an Atari Emulator, so that they could read the ram-locations holding scores in Atari games.

They build a variant of Q-learning, whose input is raw pixels and whose output is a value function estimating future rewards

They tested their method on seven Atari 2600 with no modification of the model to fit the individual game.

The method outperform all previous automatic approaches on six of the games and surpasses a human expert on three of them.



Deep Mind Atari video

<https://www.youtube.com/watch?v=V1eYniJ0Rnk>



Google DeepMind's Deep Q-learning playing Atari Breakout!



Two Minute Papers ✓
1.59M subscribers

Join

Subscribe

👍 6.8K



➦ Share

⬇ Download

💖 Thanks

🔖 Save



1.3M views · 9 years ago

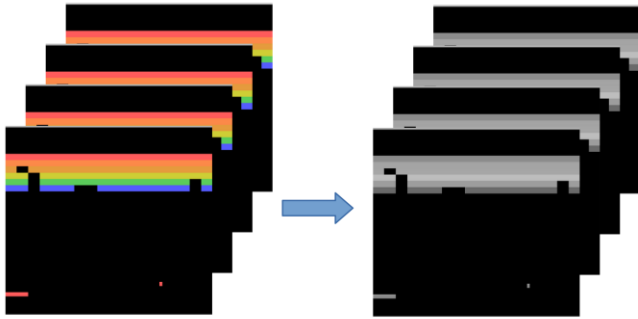
Google DeepMind created an artificial intelligence program using deep reinforcement learning that plays Atari games and improves itself to a superhuman level. It is capable of playing many Atari games and uses a combination of deep artificial neural networks and reinforcement learning. After presenting their initial results with the algorithm, Google almost immediately acquired the company for several hundred million dollars, hence the name Google DeepMind. Please enjoy the footage and let me know if you have any questions regarding deep learning! ...more

Deep Mind's DQL architecture



4 successive frames helps
the network detect
movements.

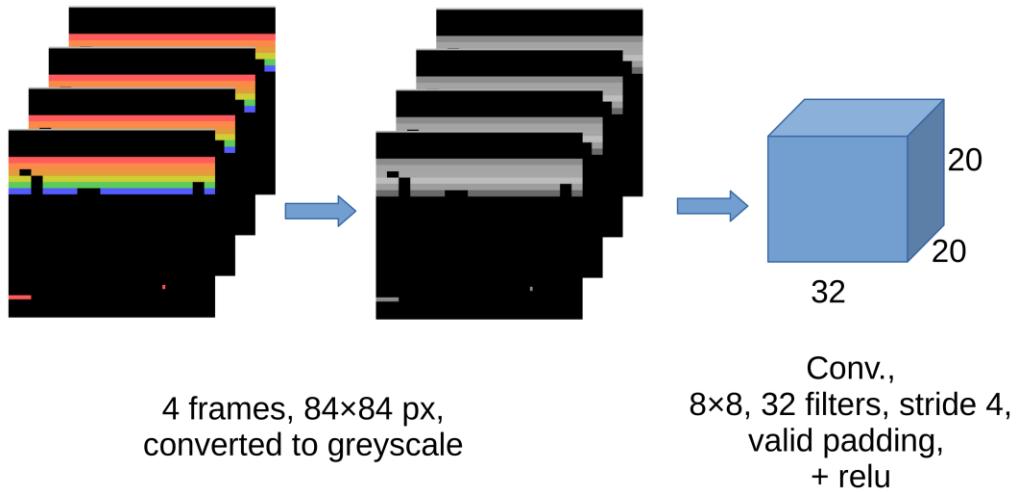
Deep Mind's DQL architecture



4 frames, 84×84 px,
converted to greyscale

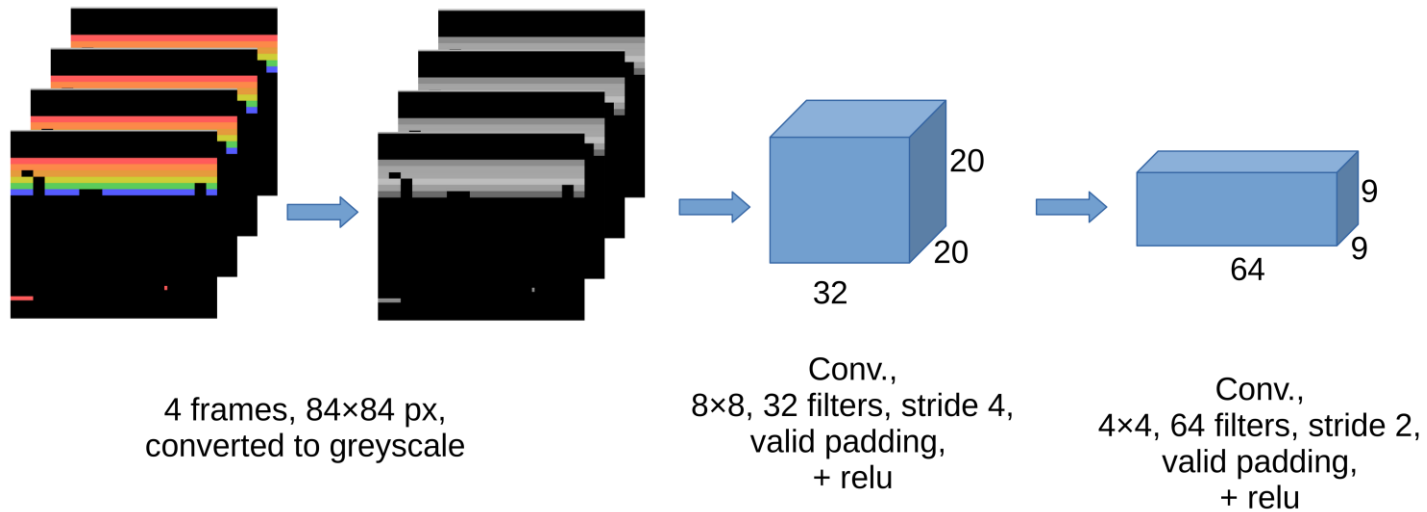
4 successive frames helps
the network detect
movements.

Deep Mind's DQL architecture



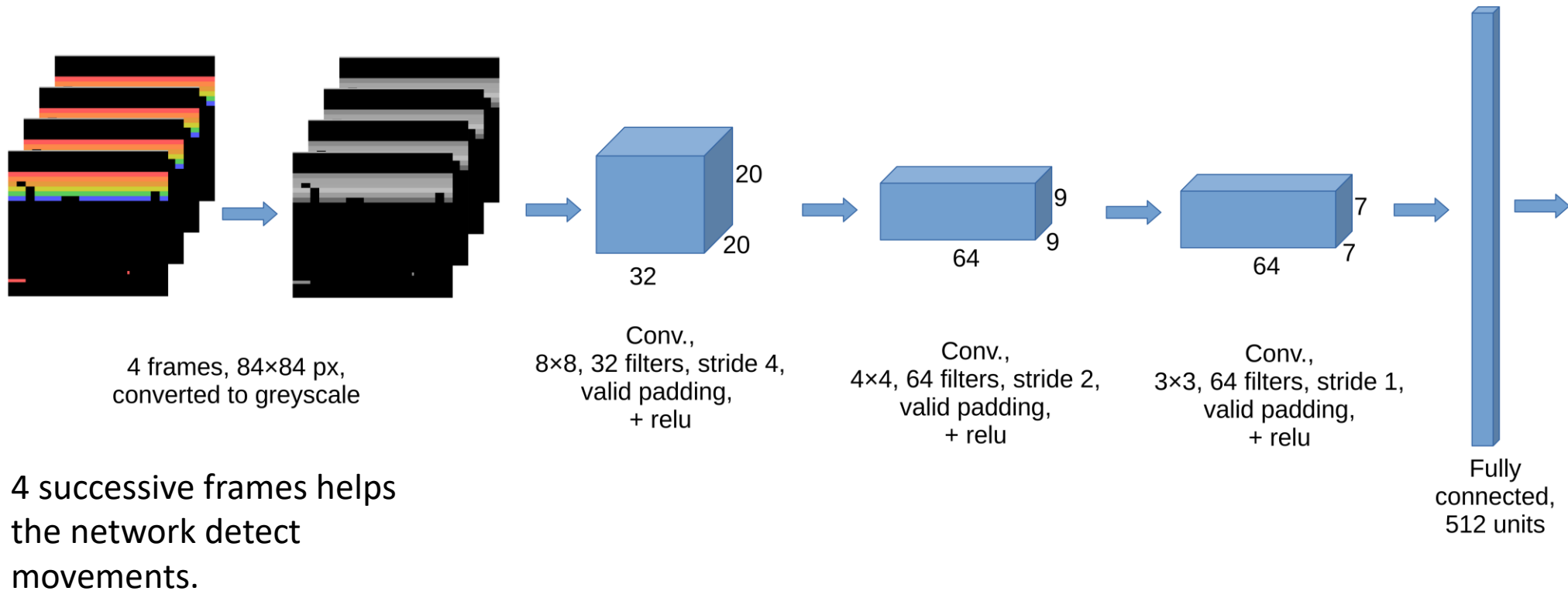
4 successive frames helps
the network detect
movements.

Deep Mind's DQL architecture

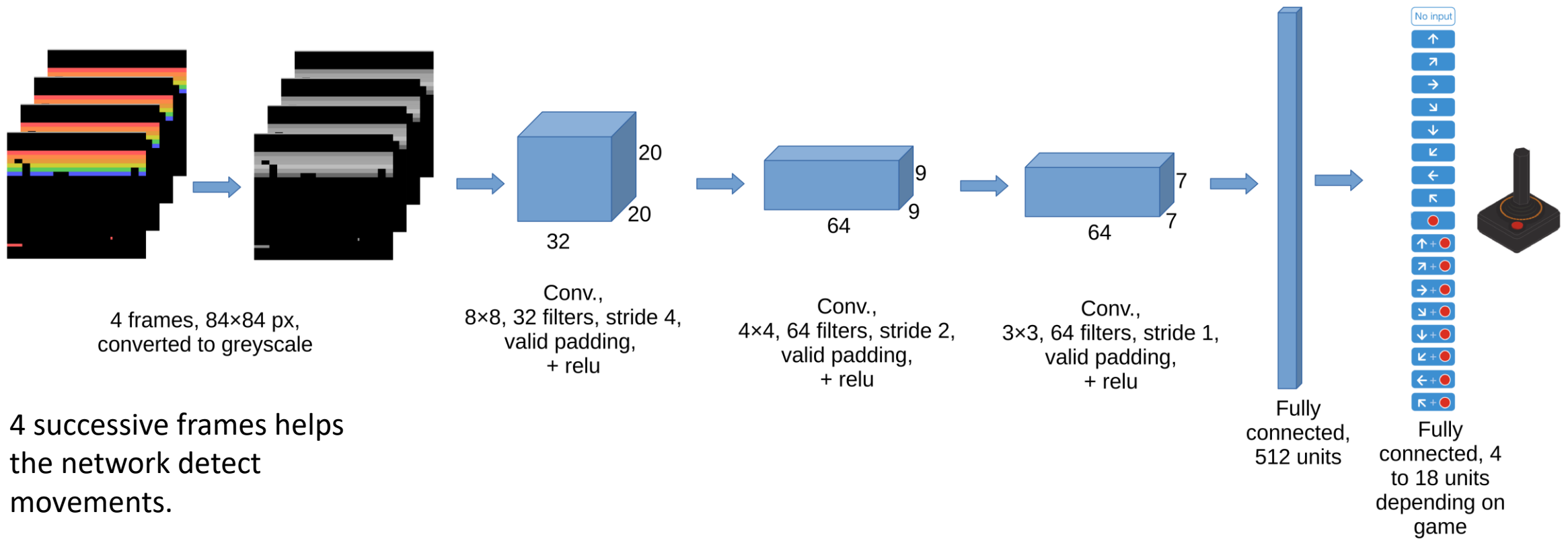


4 successive frames helps
the network detect
movements.

Deep Mind's DQL architecture

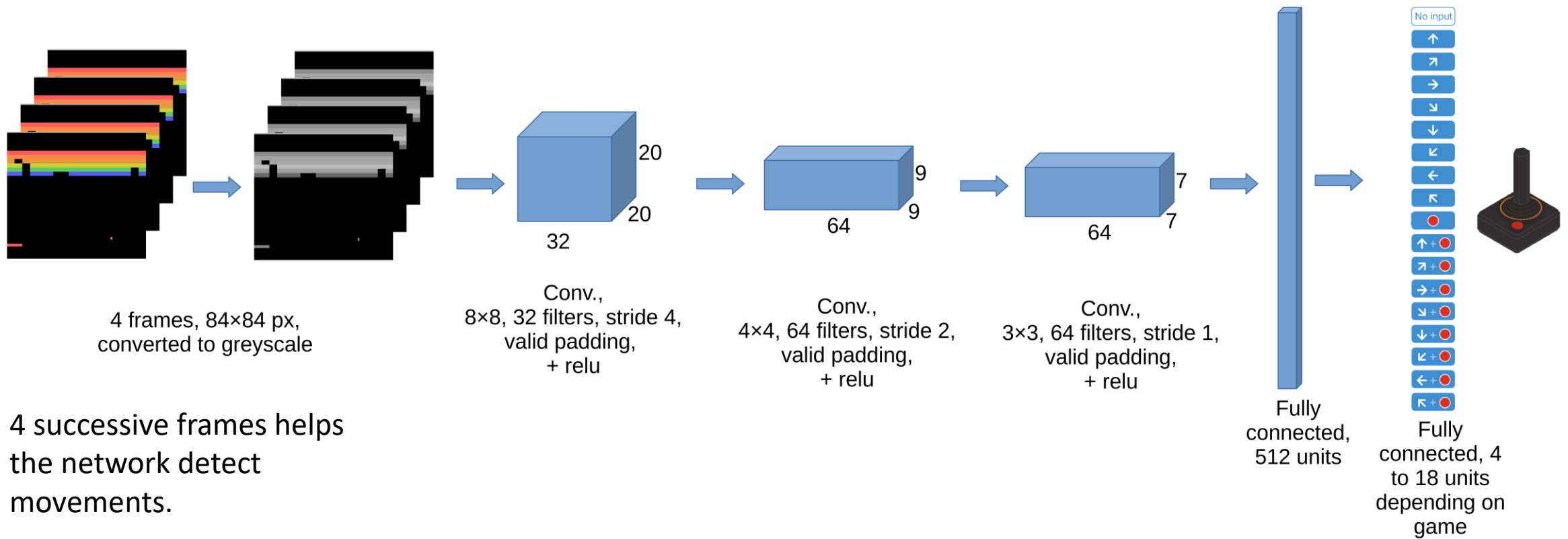


Deep Mind's DQN architecture



The outputs correspond to the predicted Q-values of the individual actions for the input state.

Deep Mind's DQN architecture



4 successive frames helps
the network detect
movements.

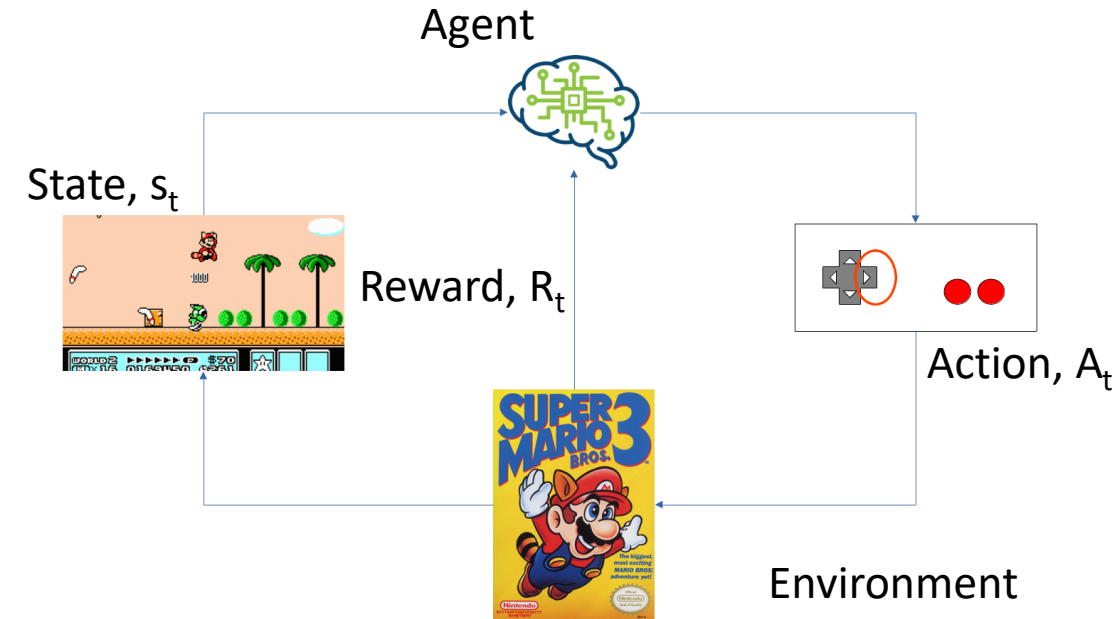
Newer research has shown
that LSTMs are better for
keeping temporal
information

The outputs correspond to the predicted Q-values
of the individual actions for the input state.

Replay memory

In some games, different levels might be very different. Therefore, training on only one level at the time will lead to the network forgetting what it has learned from other level.

Just like when we train a classifier and mix samples from all classes in minibatches, we should mix experience from the full environment.



Deep Minds' approach: Create a *replay memory* of previous experiences (state, action, reward, new state) and store it.

One experience $e_t = (s_t, a_t, r_t, s_{t+1})$

Replay memory of experiences $D_t = \{e_1, \dots, e_t\}$

During training, apply Q-learning updates on samples (or minibatches) of experience $(s,a,r,s') \sim U(D)$, drawn uniformly at random from the pool of stored samples. In Deep Minds' implementation they store the 1 million most recent frames.

Randomizing the samples breaks correlations between subsequent frames, which will lead to a higher exploration rate, since it will not learn (and repeat) from the previous frame.

Deep Q-Learning Loss function

$$L_i(\theta_i) = \mathbb{E}_{(s,a,r,s') \sim U(D)} \left[\left(\underbrace{r}_{\text{Reward}} + \underbrace{\gamma}_{\text{Discount rate}} \underbrace{\max_{a'} Q(s', a'; \theta_i^-)}_{\text{Max Q value of next state } s' \text{ and possible actions } a' \text{ at that state}} - \underbrace{Q(s, a; \theta_i)}_{\text{Current Q value}} \right)^2 \right]$$

θ_i : The parameters of the Q-network at iteration i

θ_i^- : The network parameters used to compute the target at iteration i

$(s, a, r, s') \sim U(D)$: Sample from replay memory

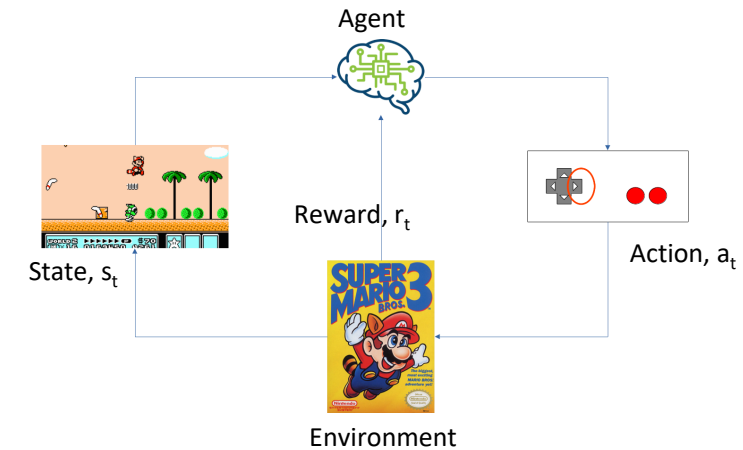
The loss is based on difference of output Q-value and a target Q-value.

We do not know the target q-value until next stage! Therefore, to know the target q-value we need to feed the next state s' into the network to get the Q values and thereby calculate the loss at the current state.

Hence, we need two passes of the network for each step.

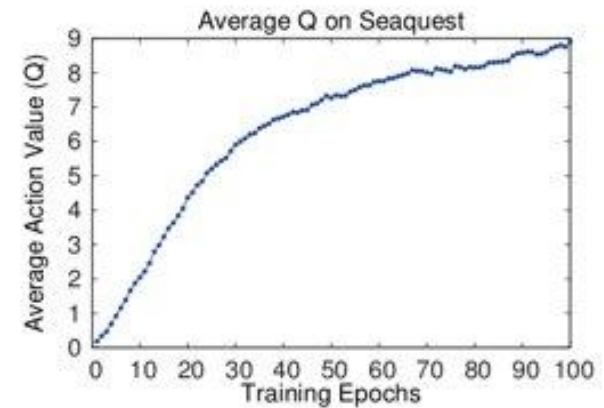
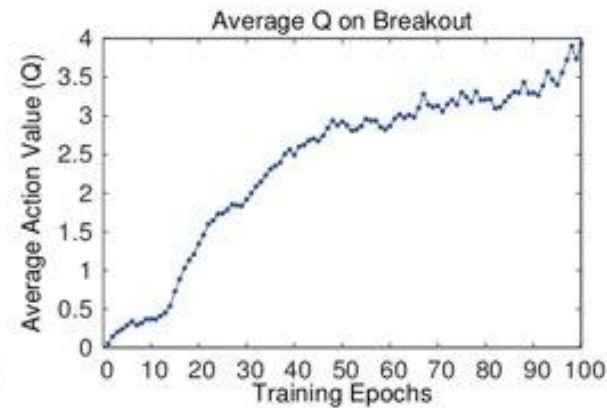
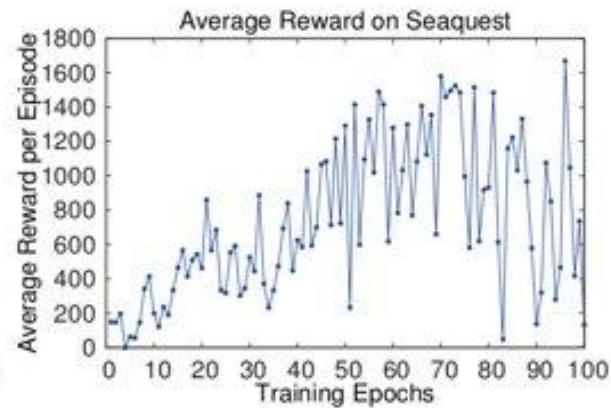
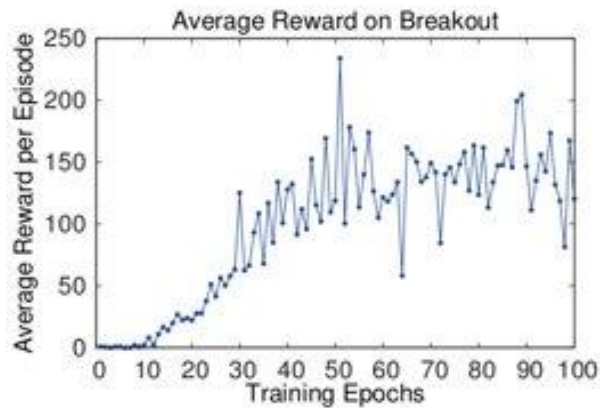
Deep Q-Network training procedure

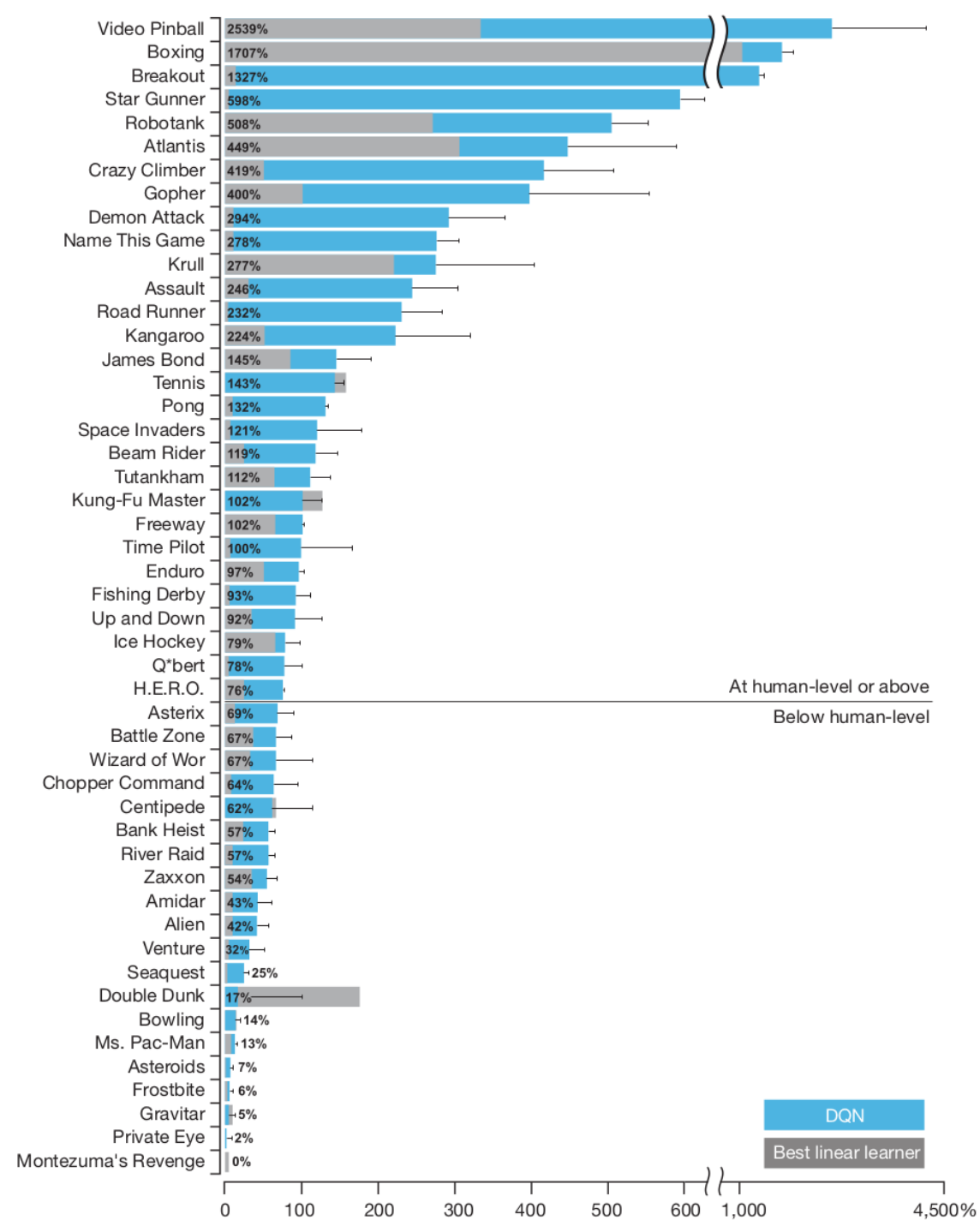
1. Initialize replay memory
2. Initialize network (random weights)
3. *For each episode:*
 1. Initialize starting state
 2. *For each time step:*
 1. Select an action
 2. Execute selected action
 3. Observe reward and next state
 4. Store experience in replay memory, $e_t = (s_t, a_t, r_{t+1}, s_{t+1})$
 5. Sample random sample (or batches) of experience(s) from replay memory. E.g., $e_4 = (s_4, a_4, r_5, s_5)$
 1. Feed s_4 and s_5 through network to calculate output Q-value at s_4 and target Q-value at s_5
 2. Calculate loss and update using gradient descent



Deep Mind's Atari Training

"In addition to seeing relatively smooth improvement to predicted Q during training we did not experience any divergence issues in any of our experiments. This suggests that, despite lacking any theoretical convergence guarantees, our method is able to train large neural networks using a reinforcement learning signal and stochastic gradient descent in a stable manner."





Comparison of the DQN agent with the best reinforcement learning methods in the literature at that time.

The performance of DQN is normalized with respect to a professional human games tester (that is, 100% level)

Moving targets

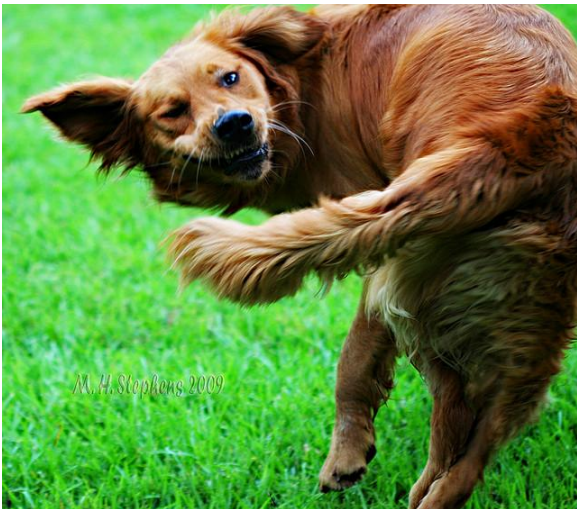


Problem with Deep Q-learning: Moving targets

In deep Q-learning we use the same parameters (weights) for estimating the target Q-values as we used to calculate the Q-value (The two steps of the network presented in the Loss-function)

Consequently, there is a big correlation between the target and the parameters (w) we are changing.

The target of the Q value at state s is approximated from the next state s' . Since s and s' share the same network and have only one step between them it's like a dog chasing its own tail: At every step, our Q-value moves closer to the target Q-value, but since the target Q-value is calculated using the same weights, the target will also move in the same direction :-)



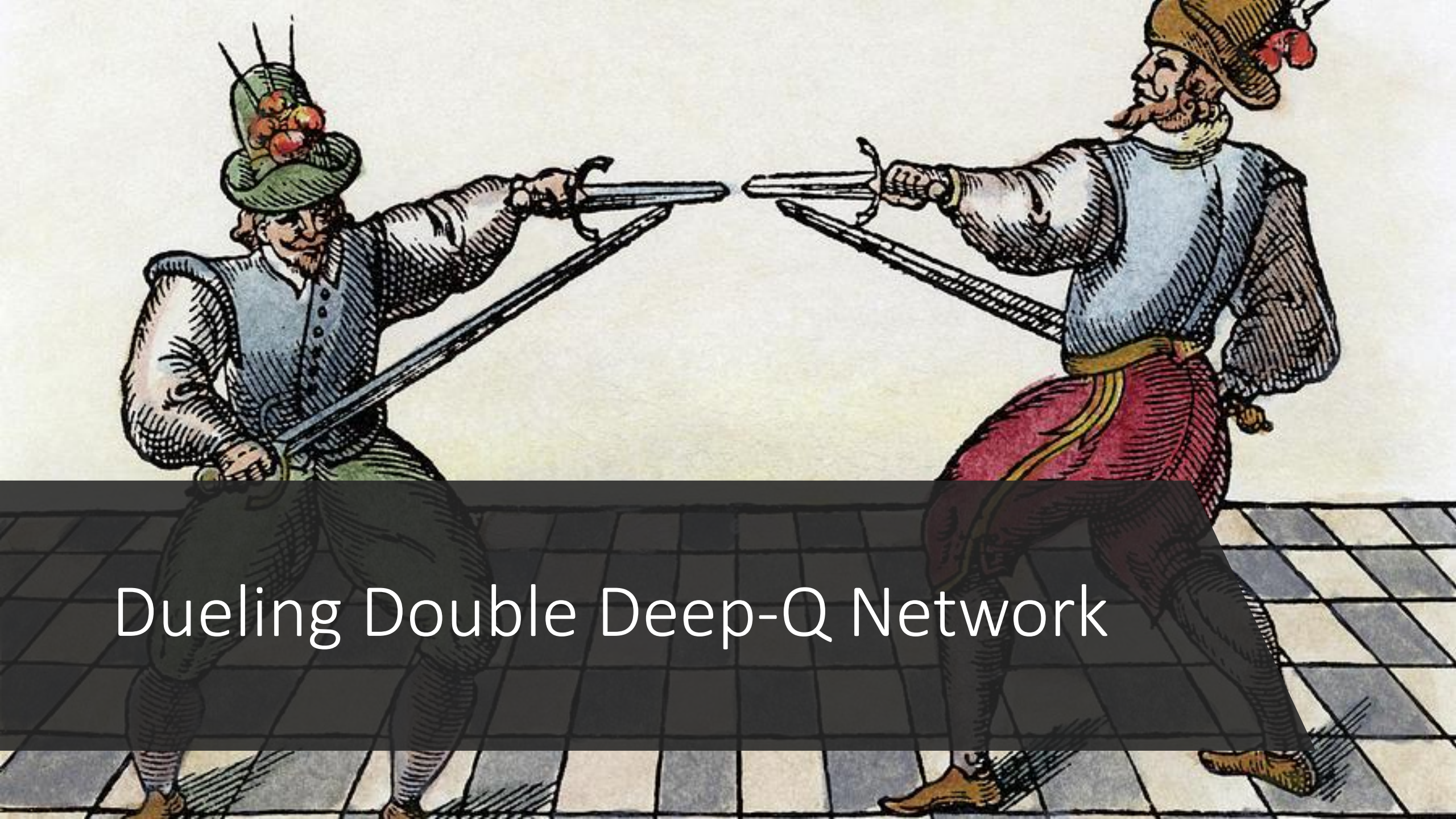
$$\Delta w = \alpha \left[\underbrace{(R + \gamma \max_a \hat{Q}(s', a, w))}_{\text{Maximum possible Qvalue for the next_state (= Q_target)}} - \underbrace{\hat{Q}(s, a, w)}_{\text{Current predicted Q-val}} \right] \nabla_w \hat{Q}(s, a, w)$$

TD Error

Gradient of our current predicted Q-value

Problem with Deep Q-learning: Moving targets

- We introduce a separate network (Target network) for estimating the Q-target rather than doing a second pass through the policy network.
 - Thereby the targets stay fixed when the policy network is updated.
- After x time-steps, we copy the parameters from our DQN network to update the target network.

A historical illustration of two men in 16th-century attire dueling with rapier swords. The man on the left wears a green hat with red ornaments and a blue vest over a white shirt. The man on the right wears a brown hat with a red plume and a blue tunic over a red skirt. They are standing on a checkered floor. A semi-transparent dark grey banner is overlaid across the middle of the image, containing the text "Duelling Double Deep-Q Network" in white.

Duelling Double Deep-Q Network

Dueling DQN (aka DDQN)

- A DQN variation

Remember that Q-values correspond **to how good it is to be at that state and taking an action $Q(s,a)$** . We can decompose $Q(s,a)$ as the sum of $A(s,a)$ and $V(s)$:

$$Q(s, a) = \underbrace{A(s, a)} + \underbrace{V(s)}$$

the advantage of taking that action at that state (how much better is to take this action versus all other possible actions at that state).

the value of being at that state

With DDQN, we want to separate the estimator of these two elements, using two new streams in the network

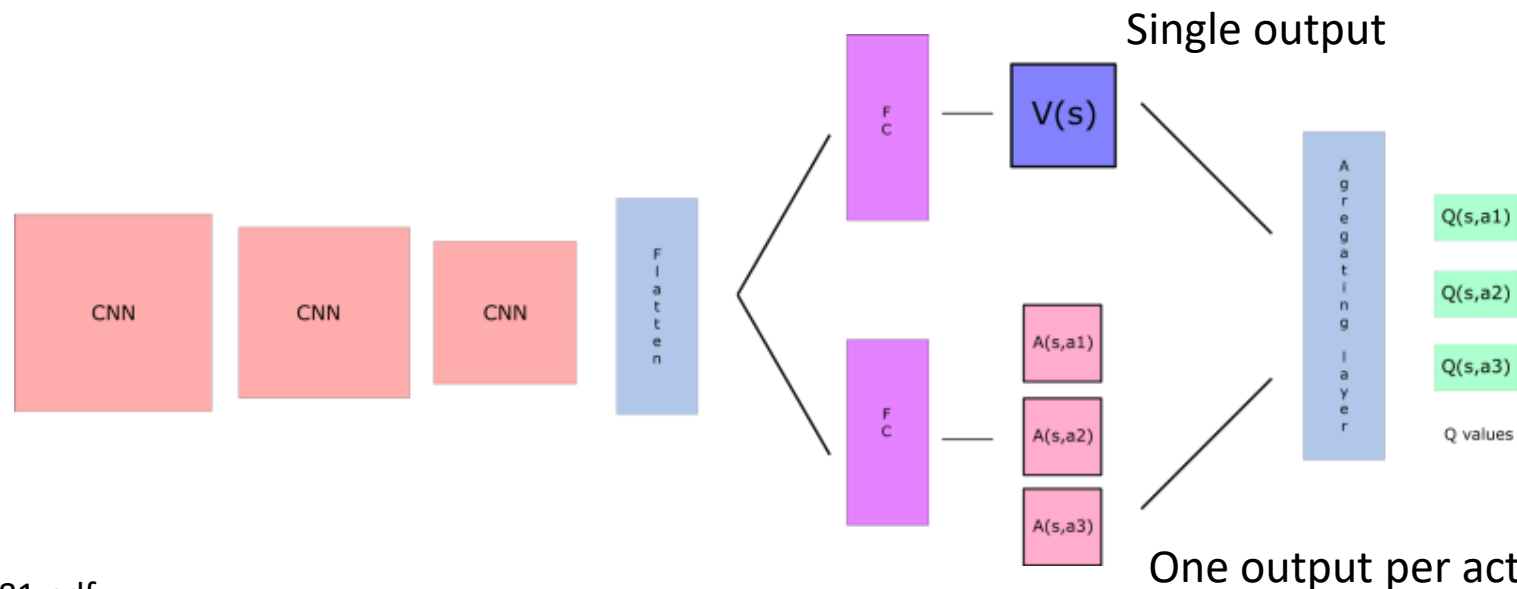
By explicitly separating the two estimators, the dueling architecture can learn which states are (or are not) valuable, without having to learn the effect of each action for each state.

Dueling DQN (aka DDQN)

$$Q(s, a) = \underbrace{A(s, a)}_{\text{the advantage of taking that action at that state (how much better is to take this action versus all other possible actions at that state)}} + \underbrace{V(s)}_{\text{the value of being at that state}}$$

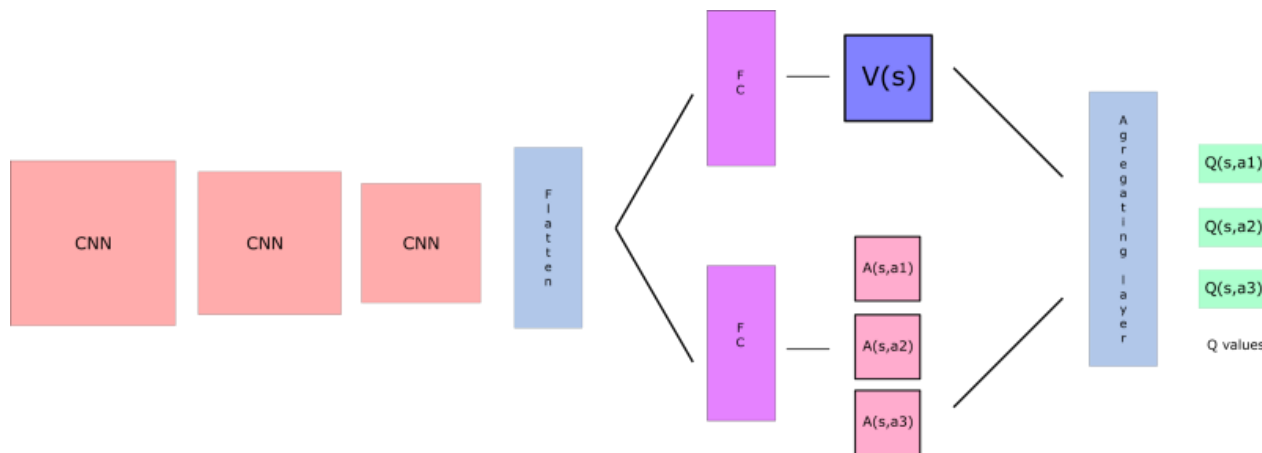
the advantage of taking that action at that state (how much better is to take this action versus all other possible actions at that state).

the value of being at that state

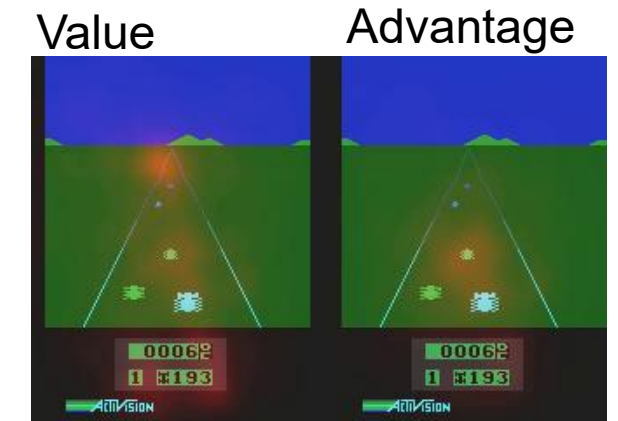
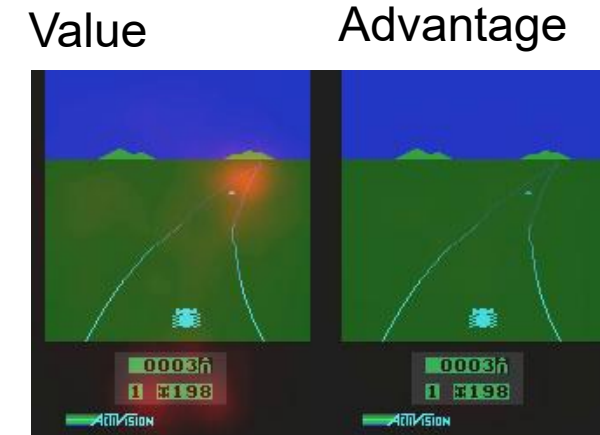


Dueling DQN (aka DDQN)

- It might seem odd that we separate action and value and then merge them.
- Intuitively, the network can now learn actions without learning values, where in standard DQN, the network had to also calculate a value
 - This is useful as some actions does not affect the value
 - For instance, moving right or left only matters if there is a risk of collision.



Value and advantage saliency maps



The value stream learns to pay attention to the road. The advantage stream learns to pay attention only when there are cars immediately in front, so as to avoid collisions.

Dueling DQN (aka DDQN) aggregation layer

To find $V(s)$ and $A(s,a)$ given $Q(s,a)$ will be a problem for our back propagation step if we just add the values. To avoid this problem, we can force our advantage function estimator to have 0 advantage at the chosen action.

Difference between "A"s is often small compared to their amplitude. Therefore, we subtract the mean

Aggregation layer:

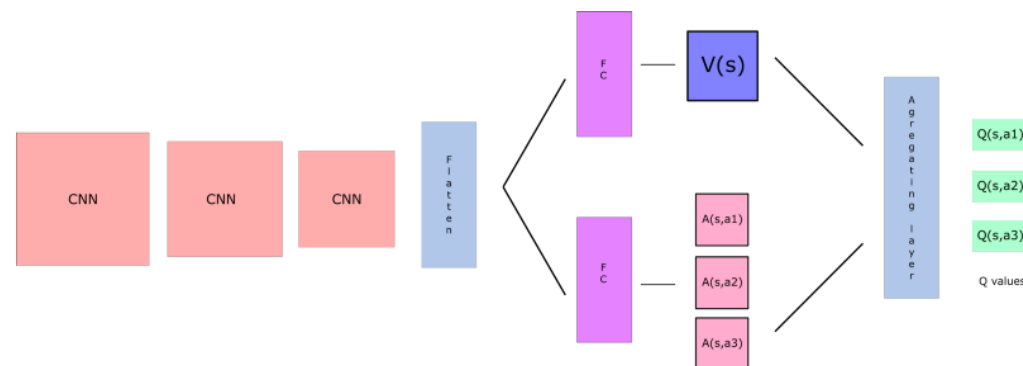
$$Q(a, s, \theta, \alpha, \beta) = V(s; \theta, \beta) + \left(A(s, a; \theta, \alpha) - \frac{1}{|\mathcal{A}|} \sum_{a'} A(s, a'; \theta, \alpha) \right)$$

Parameters of common network Parameters of Value stream
Parameters of Advantage Stream Average advantage of possible actions

How much better the advantage of an action is than other advantages of actions. Works as regularization that keeps outputs of advantage stream around 0

we choose the optimal action a^* based on:

$$a^* = \arg \max_{a' \in \mathcal{A}} Q(s, a'; \theta, \alpha, \beta)$$



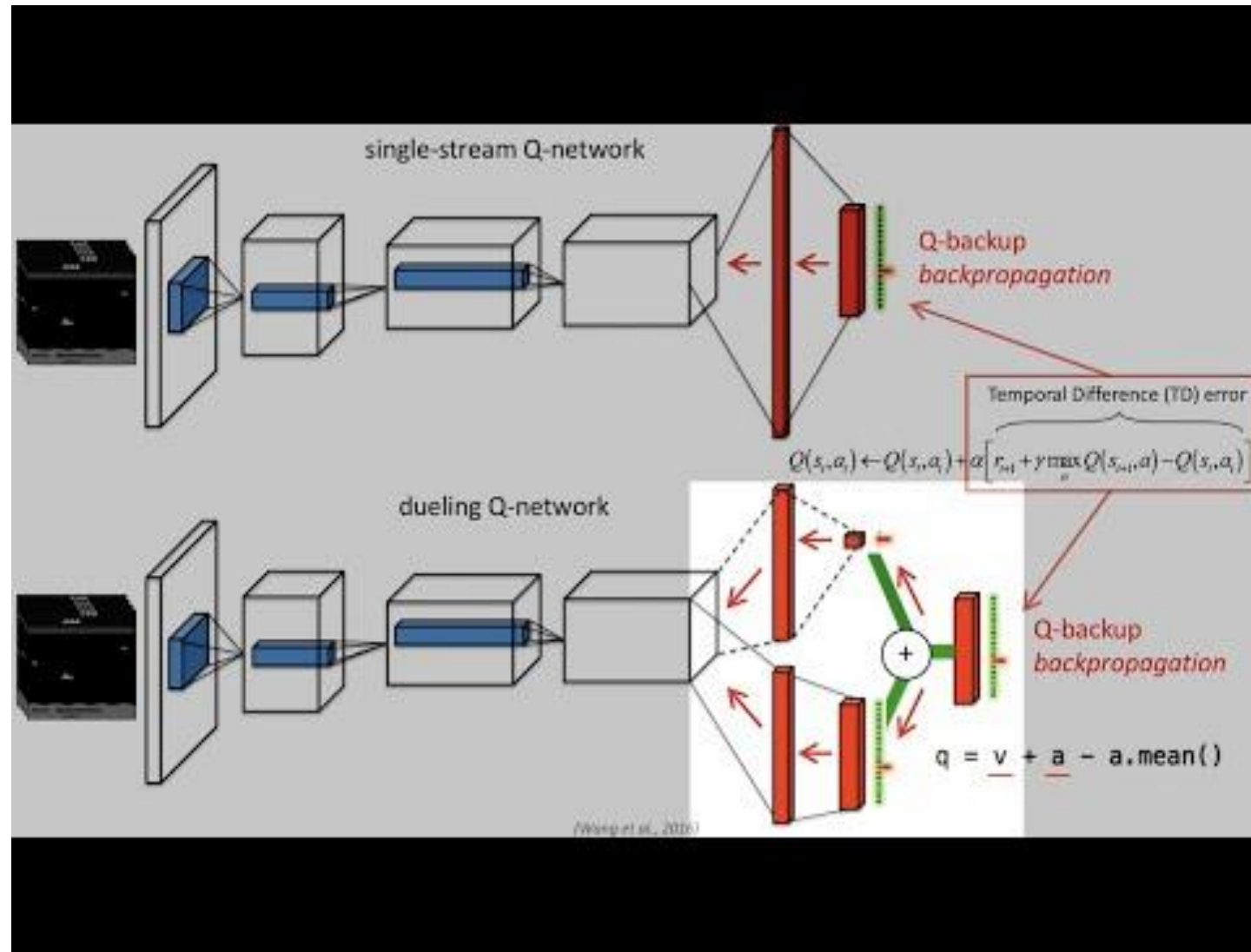
Training Dueling DQN

- The Dueling DQN shares the same input and output as standard DQN, and, therefore, the training procedure is identical:
 - Calculate loss based on the difference between output Q-value and target Q-value

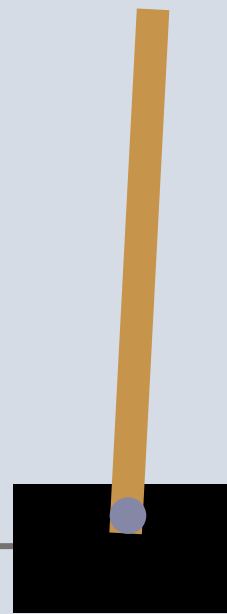
$$L(\theta) = \frac{1}{N} \sum_{i \in N} (Q_{\theta}(s_i, a_i) - Q'_{\theta}(s_i, a_i))^2$$

$$\text{where } Q'_{\theta} = R(s_t, a_t) + \gamma \max_{a'_i} Q_{\theta}(s'_i, a'_i)$$

Video that takes us through the implementation of DDQN
<https://youtu.be/XjsY8-P4WHM>

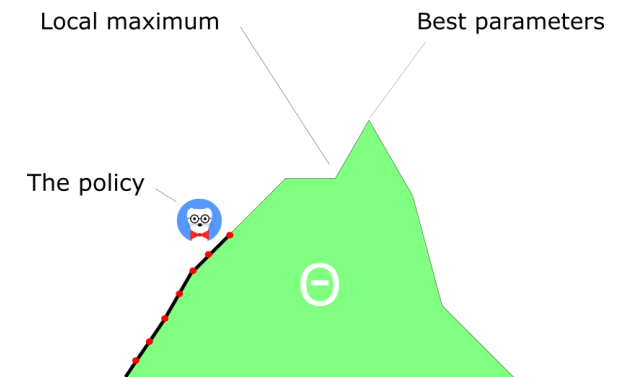


Policy gradient



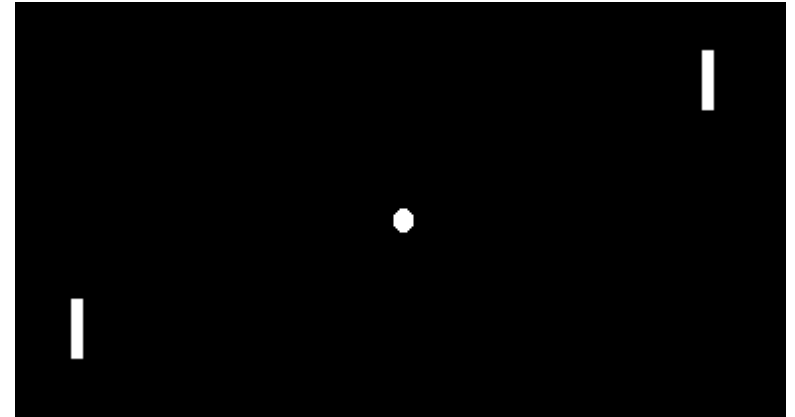
Policy Gradient

- Policy Gradients is another way to solve RL problems
- Policy gradients is **like supervised learning**, but on a continuously changing dataset (the episodes)
- Minor differences between PG and supervised learning:
 1. We don't have the correct labels y_i so as a “fake label” we substitute the action we happened to sample from the policy when it saw x_i with the outcome of the episode.
- Benefit of Policy Gradient over DQN:
 - PG tends to have better convergence properties
 - With PG we are guaranteed to end at least on a local maximum
- Policy gradients are more **effective in high dimensional action spaces**
 - E.g., self-driving car where steering wheel can be turned in continuous angles



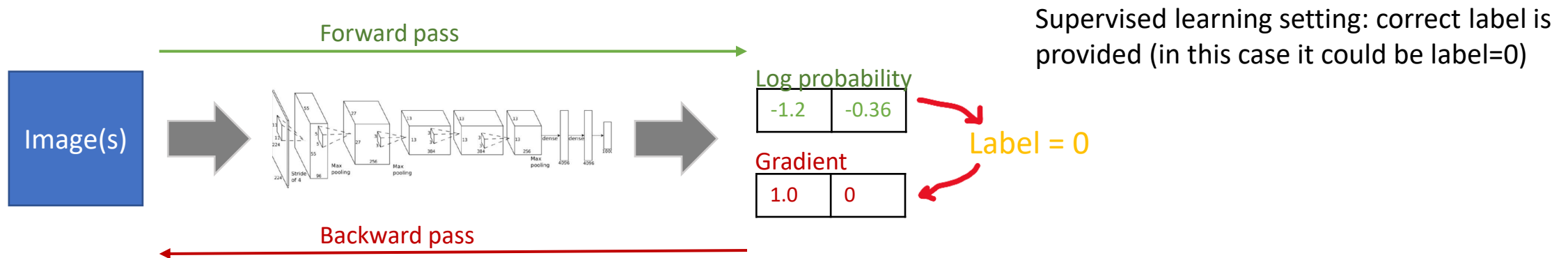
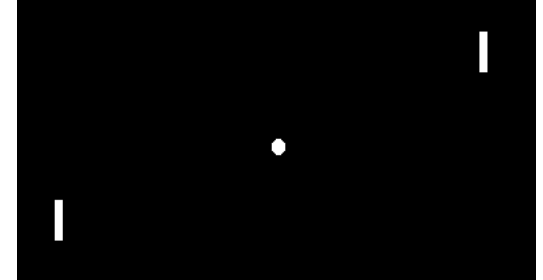
Let's consider the game PONG

- Goal is to get the ball past the opponent
- You have two controls: move **UP** or move **DOWN**
- First, we define a policy network that implements our player (or “agent”).
 - This network will take the state of the environment and decide what we should do. E.g. move UP or DOWN in a game



Training PONG in a Supervised setting

- How to get to the reward in Policy Gradients? Let's start by looking at a supervised setting: We have a model and a set of labelled actions (0=up, 1=down):
- Output is fed through a softmax function to give us "probabilities"
- We always optimize the log probability of the correct label (this makes math nicer, and is equivalent to optimizing the raw probability because log is monotonic)
- Policy network outputs probability of going UP as 30% (logprob -1.2) and DOWN as 70% (logprob -0.36)

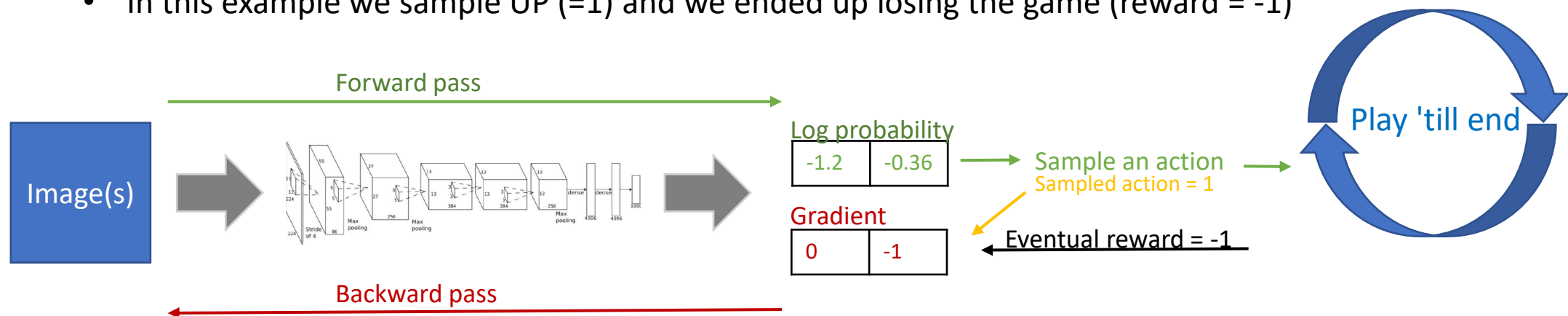
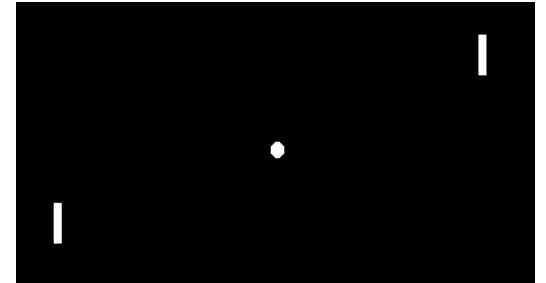


Run backprop to compute the gradient vector and tell how much our weights should be changed. s =state (input image), a = network output (action)

$$\nabla_w \log p(a = \text{UP} | s)$$

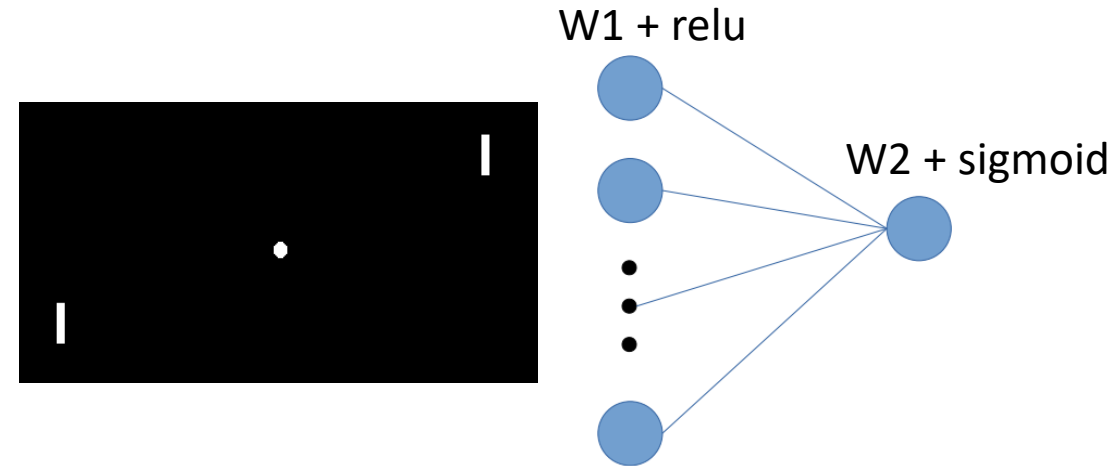
Policy Gradient - PONG

- However, we do not have labelled data!
- The policy network outputs probability of going UP as 30% (logprob -1.2) and DOWN as 70% (logprob -0.36). We will now sample from this distribution (a biased coin) and perform the sampled action.
- Assume we sample "DOWN". Since the game is not finished, we cannot tell whether this was a good move and what the gradient should be.
- We, therefore, have to wait until the game ends and use the reward we get (either +1 if we won or -1 if we lost).
 - I.e., actions that happen to lead to good outcomes get encouraged in the future, and actions taken that lead to bad outcomes get discouraged.
 - In this example we sample UP (=1) and we ended up losing the game (reward = -1)



Policy Gradient training:

- Assigning reward to actions



Assume we have the following network:

```
h = np.dot(W1, x)    # compute hidden layer neuron activations
h[h<0] = 0           # ReLU nonlinearity: threshold at zero
logp = np.dot(W2, h)  # compute log probability of going up
p = 1.0 / (1.0 + np.exp(-logp))  # sigmoid function (gives probability of going up)
```

... and that each game has 200 frames.

After playing a number of games (let's say 100 games), i.e., 20 000 actions.

We can now label the outputs of each of the 20 000 actions as GOOD or BAD depending on whether we won the game.

Let's say we won 6 games and lost 94 games. We will now take the 200*6 actions and do a positive update (i.e., setting the gradient to +1) and do a backprob to encourage the network to do similarly.

We do the same for the 94 lost games but set the gradient to -1.

This is repeated with a network that is now slightly improved.

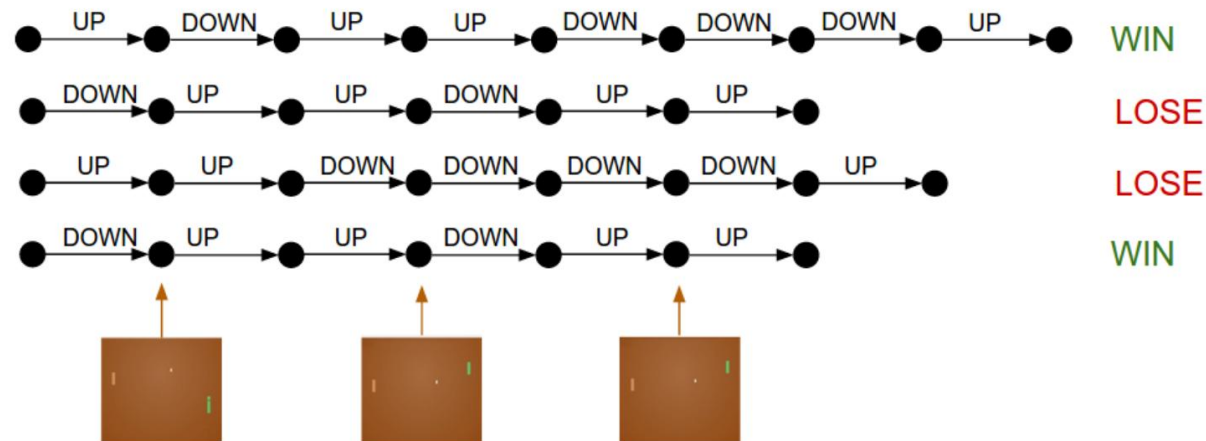
the action we sampled
input sample (frame)

$$Loss_{PONG} = \sum_i A_i \log p(y_i | x_i)$$

Advantage (in pong -1 or 1)

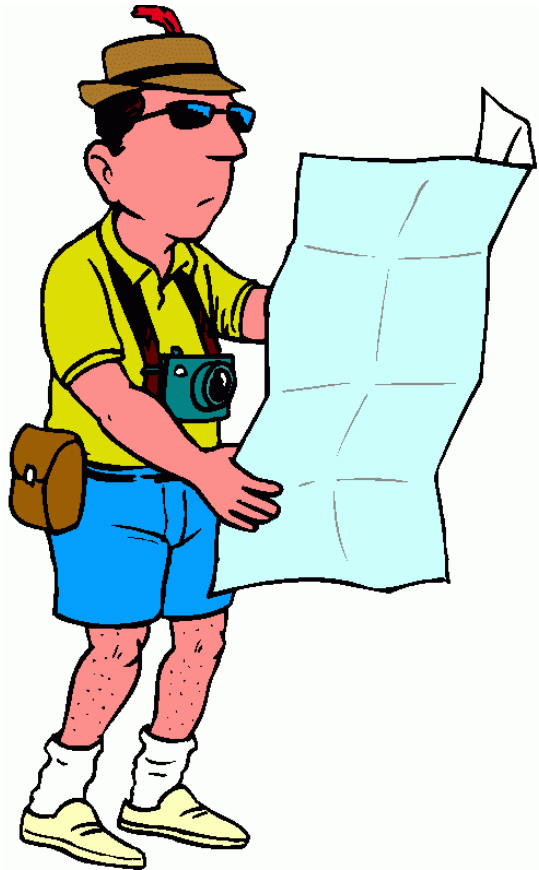
Credit assignment problem

- We get a state (e.g., a set of images. Choose an action based on the current policy (l.e. network output), and after repeating this many times, we might get a reward. But which of the previous actions contributed to this reward? Was it something we did just now? Or maybe 100 frames ago?
- In PONG: If you win, it is because you send the ball on a good trajectory. All actions since then did not influence the result.
 - The reason why you won might be an action 20 frames ago.
- What if we fail? Does that mean that the full sequence that leads to failure is bad? No.
 - Updating the full sequence will discourage the good actions taking in the beginning of the game, but we will typically update the network thousands or millions of times, so on average you'll see more positive than negative updates for the correct bounce in the beginning of the game, since a good bounce in the beginning makes winning slightly more likely



Path to reward is too complex

--> Reward Shaping



- In traditional supervised image classification, we have a target for each image. In reinforcement learning, we have a very sparse reward setting.
- In complex systems, doing random exploration will newer give positive rewards
- The process of manually designing a reward function, that guides our policy to a desired behavior.
 - Typically implemented as sub-rewards.
 - e.g., in computer game: Final goal is to win, sub-goal is to defeat enemy
- Downside:
 - You will have to design a reward function for every domain.
--> doesn't scale well
 - The policy might learn to just update the subgoal, but newer generalizing to the intended subgoal
 - In some cases, it is not desirable (e.g. playing GO), since you want the computer to not behave as humans.

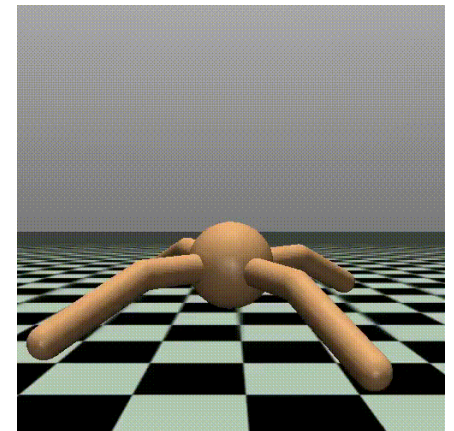
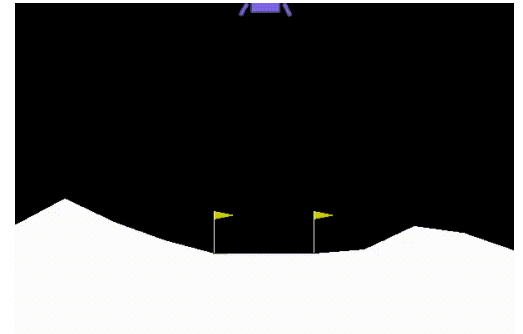
Reinforcement Learning in Python

"Stable baselines3" and "Deep Reinforcement Learning with Pytorch" and "Spinning Up" are libraries that implements common RL algorithms

- <https://stable-baselines3.readthedocs.io/en/master/>
- <https://github.com/sweetice/Deep-reinforcement-learning-with-pytorch>
- <https://spinningup.openai.com/en/latest/index.html>

GYM is a toolkit for developing and comparing reinforcement learning algorithms

- <https://gym.openai.com/>
- Install using `pip install gym`
- If you have license to download a copy of Atari games, you can install the Atari package using:
- `pip install gym[atari]`



Exercises:

- Try implementing Q-learning with Numpy using the tutorial at <https://youtu.be/q2ZOEFAaal0?t=152>
- Exercise 10.1: DeepQLearning example – understand and run the code. Then experiment with:
 - number of episodes
 - discount factor (e.g., try 0.95 - 0.99)
 - learning rate
 - network layers
 - Also, identify the two networks. What are their purpose?
- Exercise 10.2: Policy gradient example – understand and run the code, and then experiment with:
 - number of episodes
 - discount factor
 - learning rate
 - network layers
 - Choose another GYM-task <https://gym.openai.com/>

Exercises are available on brightspace page Week 10!

▼ Week 10 -
Reinforcement Learning

📄 Slides

🔗 Exercise 10.1: Deep Q-
Learning

🔗 Exercise 10.2 PG pong